

微型计算机原理

第三章 8086CPU的寻址方式和指令系统



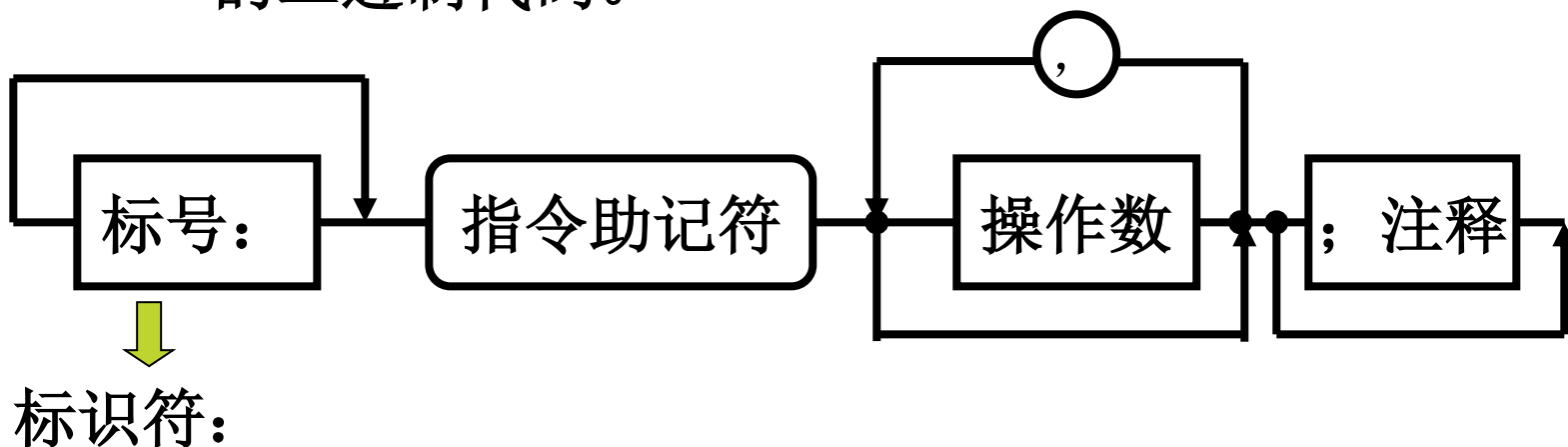
刘剑锋



2025年2月20日星期四

3.1.1 86系列汇编语言指令语句格式

汇编语言：符号化了的机器语言，用助记符代替机器语言的二进制代码。



- a~z、A~Z、0~9或某些特殊符号（@，-，？）
- 第一个字符必须是字母或某些特殊符号（@，-，？）
- ？不能单独使用
- 标识符有效长度为31个字符

例： MOV AL, 80H

3.1.2 86系列CPU的寻址方式

1. 立即寻址

操作数就包含在指令当中

例: MOV AL, 80H ; 将十六进制数80H送入AL
 MOV AX, 1090H ; 将1090H送入AX

目的操作数

源操作数

- 立即数可以是8位、16位
- 立即数只能是整数，不能是小数、变量或其它类型的数据
- 立即数只能作为源操作数

2. 寄存器寻址

386以上

操作数在8位、16位（或32位）通用寄存器中

寄存器可以是AX、BX、CX、DX、SI、DI、SP、BP
AH、AL、BH、BL、CH、CL、DH、DL

例：

INC	CX	； 将CX的内容加1
ROL	AH, 1	； 将AH中的内容循环左移一位
MOV	AX, BX	
MOV	AX, 1090H	

在一条指令中，可以对源操作数采用寄存器寻址，也可以对目标操作数采用寄存器寻址，还可以两者都采用寄存器寻址方式

3 几种存储器寻址方式

①直接寻址

操作数的有效地址包含在指令中

例：

MOV AX, [1070H] ; DS段的1070H和1071H两单元的内容传送到AX中

物理地址 = DS × 16d + EA

若在其他段，必须给出段跨越前缀。

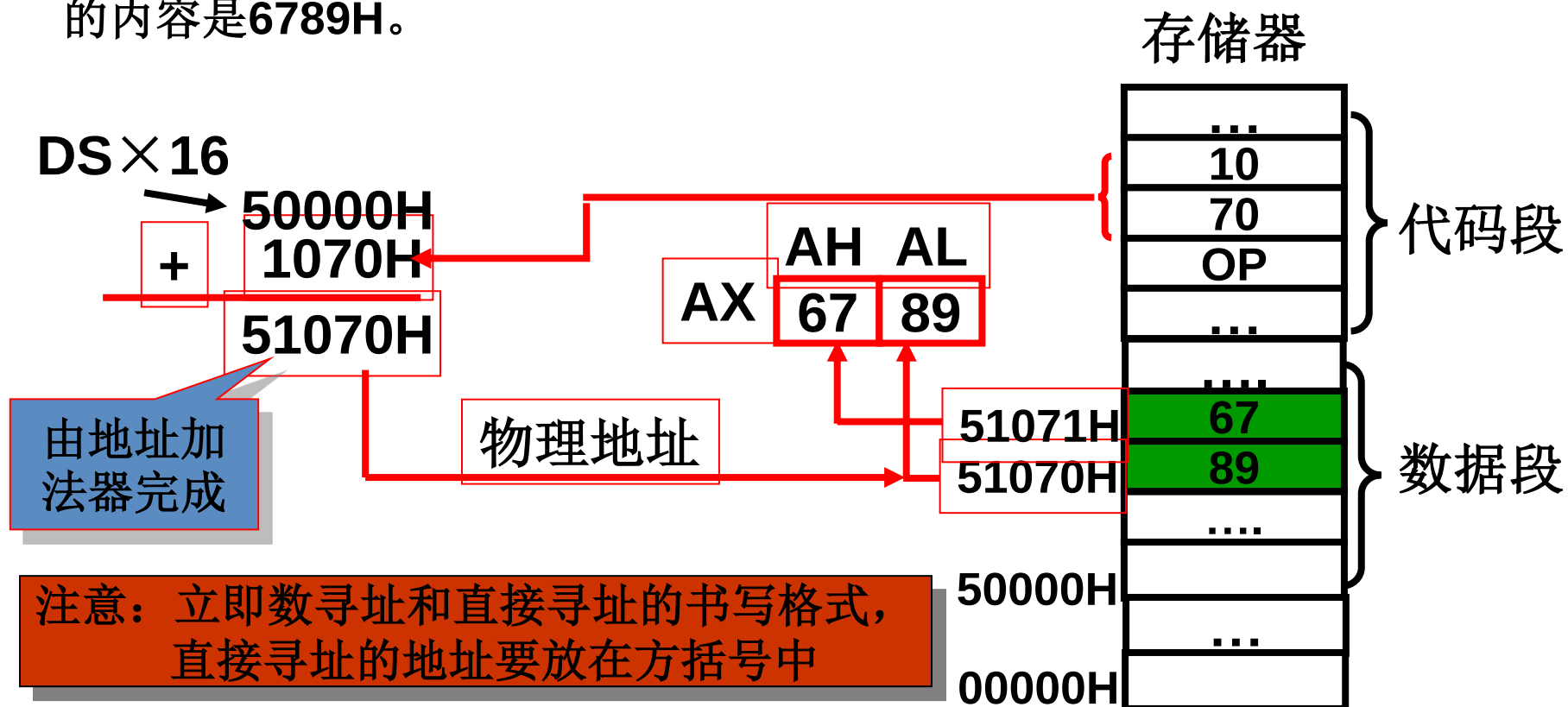
MOV AX, ES:[500H] ; EA = ES × 16d + 500H

①直接寻址

例：

MOV AX, [1070H] ； DS段的1070H和1071H两单元的内容传送到AX中

假设DS=5000H，地址为51070H字存储单元的内容是6789H。



3 几种**存储器**寻址方式

**BX、BP、SI、
DI 之一**

②寄存器间接寻址

——操作数在存储器中，操作数的逻辑地址由**寄存器**提供，寄存器的内容是**段内的偏移地址**。

注：如果没有特殊说明，用寄存器SI、DI和BX间接寻址时，对应的段寄存器是DS。如果用寄存器BP间接寻址时，对应的段寄存器是SS。

例1:

MOV AX, [SI] 假设 (DS) = 5000H, (SI) = 1234H

则：源操作数的物理地址是51234H

**再假设51234H字单元的内容是6789H，执行该条指令后
(AX) = 6789H**

3 几种存储器寻址方式

MOV AX, 1234H[DI]

③寄存器相对寻址

——带位移量的寄存器间接寻址

↑
另一种书写格式

带位移量的寄存器间接寻址

$$(有效地址) EA = \begin{Bmatrix} [BX] \\ [BP] \\ [SI] \\ [DI] \end{Bmatrix} + \begin{Bmatrix} 8位位移量 \\ 16位位移量 \end{Bmatrix}$$

例1:

MOV AX, [DI+1234H] 假设 (DS) = 5000H, (DI) = 3678H

则: 源操作数的物理地址是548ACH →

再假设548ACH字存储单元的内容是55AAH

那么, 执行该指令后, (AX) = 55AAH

$$\begin{array}{r} 5000H \\ 3678H \\ + 1234H \\ \hline 548ACH \end{array}$$

3 几种存储器寻址方式

④基址变址寻址方式

有效地址EA 是一个基址寄存器和一个变址寄存器的内容之和，两寄存器均由指令指定，如基址寄存器为BX时，段使用DS；基址寄存器为BP时，段使用SS。

$$PA = 16d \times (DS) + (BX) + (SI) \text{ 或 } (DI);$$

当使用BP时 $PA = 16d \times (SS) + (BP) + (SI) \text{ 或 } (DI)。$

例1:

MOV DX,[BX][SI]

也可以写为 **MOV DX, [BX+SI]**

如果(DS)=2000H, (BX)=0100H, (SI)=3000H,
(23100H)=0A1H, (23101H)=00H,

则 $PA=20000H+EA=20000H+0100H+3000H=23100H$ ，指令执行后DX内容为00A1H。

3 几种存储器寻址方式

⑤相对基址变址寻址方式

有效地址 $EA = (\text{基址寄存器}) + (\text{变址寄存器}) + (8 \text{ 或 } 16)$ 位位移量。

当基址寄存器为BX时，使用DS段寄存器；

当基址寄存器为BP时，使用SS段寄存器。

变址寄存器为SI或DI。

例1：

MOV AL, MASK[BX][DI] 也可以写为MOV AL, MASK [BX+DI],
或MOV AL, [MASK+BX+DI], MASK为位移量。

若(DS)=1000H, (BX)=0200H, (DI)=3000H,

MASK=0040H, 物理地址(13240H)=35H,

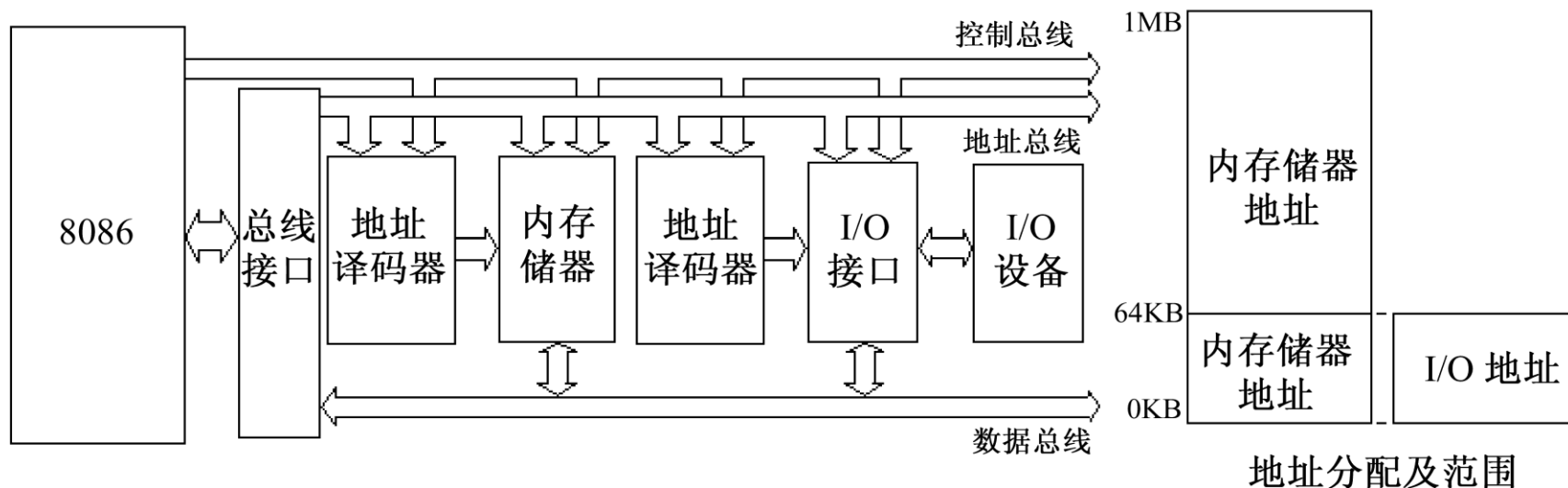
则 $PA = 16d \times (DS) + (BX) + (DI) + MASK = 13240H$,

则指令执行后AL的内容为35H。

8086偏移地址和段寄存器之间默认的组合

段寄存器	偏移地址	用途
CS	IP	指令地址
SS	SP	堆栈操作地址
SS	BP、SI、DI 或指令中给出的8/16位数据（相对量）	存取堆栈内的数据地址
DS	BX、SI、DI 或指令中给出的8/16位数据（相对量）	存取数据段的数据地址
ES DS	DI ，串指令中使用（目标串地址） SI ，串指令中使用（源串地址）	串操作的目标操作数地址

5. 8086独立的I/O寻址



8086有相互独立的访问存储器和访问**I/O**端口的指令，在执行存储器或**I/O**指令时，**8086**引脚上呈现不同的信号。**8086**在执行访问**I/O**端口的输入输出指令时，读写信号或有效，同时**M/信号**输出低电平。在执行访问存储器指令时，除了读写信号或有效外，**M/信号**输出高电平。这样就区分了**I/O**读写和存储器读写。最大模式下可以通过外部逻辑电路产生对**I/O**端口的读/写信号。**8086**的**I/O**系统是单独编址的。

8086允许使用16条地址线作，寻址范围是64KB，可编址65536个8位I/O端口。地址为0000-FFFFH，与存储器空间相互独立。

两个相邻编号的8位端口也可以组合成一个16位端口。指令系统中既有访问8位端口的输入/输出指令，也有访问16位端口的输入/输出指令。另外，可寻址空间为64K个8位的I/O端口。

(1) 直接寻址的输入/输出指令（当I/O端口的地址为0000---00FFH）

格式：IN AL, PORT 或 IN AX, PORT
OUT PORT, AL 或 OUT PORT, AX

例如：

IN AL, 50H ; 将50H端口的字节读入AL
IN AX, 70H ; 将70H、71H两端口的值读入AX
OUT 70H, AL ; 将AL的字节值输出到70H端口

(2) 间接寻址的输入/输出指令（当I/O端口的地址为0100H--FFFFH）

格式：IN AL, DX 或 IN AX, DX
OUT DX, AL 或 OUT DX, AX

例如：

IN AL, DX ; 从DX所指的端口中读取一个字节
IN AX, DX ; 从DX和DX+1所指的两个端口读取一个字
OUT DX, AL ; 将AL的字节值输出到DX所指的一个端口



3.2 8086微处理器指令系统



8086微处理器指令系统

8086 CPU指令系统，按功能可以分为6类：

1. 数据传送指令
2. 算术运算指令
3. 逻辑指令
4. 串处理指令
5. 控制转移指令
6. 处理器控制指令

8086有4种数据传送指令，用于实现：

- 寄存器之间
- 存储器与寄存器
- 累加器AX或AL与I/O端口
- 立即数到寄存器或到存储器的字节或字的传送

3.2.1 通用数据传送指令



1. 最基本的传送指令

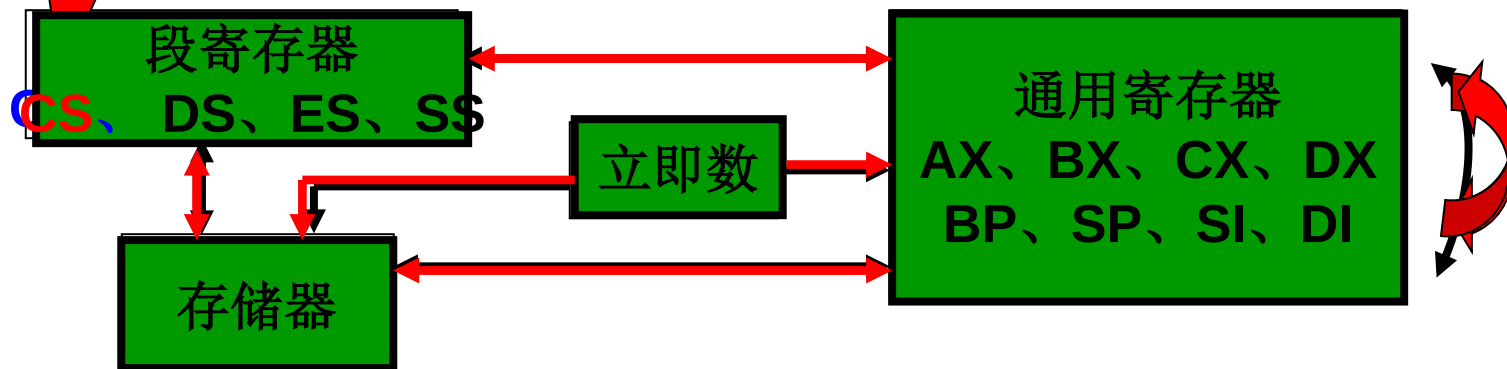
目的
↓

源
↓

格式: **MOV OPRD1, OPRD2**

功能: 把一个字节或一个字从源操作数**OPRD2**送至目的操作数**OPRD1**, 源操作数不改变。

CS不能做目的的操作数



MOV指令数据传送方向示意图

注意: 源和目的操作数的类型要一致, 即同时为字节或同时为字



在使用这些指令时，**应注意以下几点：**

(1) **CS和IP**两个寄存器**不能作为目的操作数**，也就是CS、IP不能用MOV指令来修改。

(2) 当采用**BX、DI、SI**来间接寻址时，默认的段寄存器为**DS**；当用**BP**来寻址时默认的段寄存器为**SS**

(3) 所有**MOV指令**都不影响标志位。

(4) **不能用上述命令来实现存储单元之间、立即数至段寄存器及段寄存器之间的传送。**

表 A-1 MOV 指令

指 令 码		功 能	举 例
助记符	目的操作数、源操作数		
MOV	MEM/REG,REG/MEM	用于寄存器和寄存器/存储器之间的传送，8位或16位数据	MOV AX,CX MOV [BX],DX MOV SI,[SI]
MOV	AC , MEM	用于从存储单元传送8/16位数据到累加器	MOV AL,[1234]
MOV	MEM , AC	用于从累加器传送8/16位数据到存储单元	MOV [1234H],AX
MOV	SEG , MEM/REG	从寄存器/存储单元传送一个16位数据到段寄存器中	MOV SS,DX MOV DS,[1234H]
MOV	MEM/REG , SEG	从段寄存器传送一个16位数据到存储单元/寄存器中	MOV [2000H],DS MOV AX,DS
MOV	MEM/REG , DATA	将8/16位立即数传送到存储单元/寄存器中	MOV [BX+8],2345H MOV BX,1234H

例：

MOV AX, 1000H

； 立即数送到AX

MOV AL, ' E'

； 立即数送到AL，E的ASCII码

MOV AX, 2[BP][SI]

； 存储单元的内容送到AX

如果BP换成BX？

$(SS) \times 16 + (BP) + (SI) + 2$

判断对错

MOV	5, AL	✗
MOV	[BX], [SI]	✗
MOV	DS, 1000H	✗
MOV	CS, AX	✗
MOV	AX, [DX]	✗
MOV	SI, [SI]	✓
MOV	[SI], AX	✓

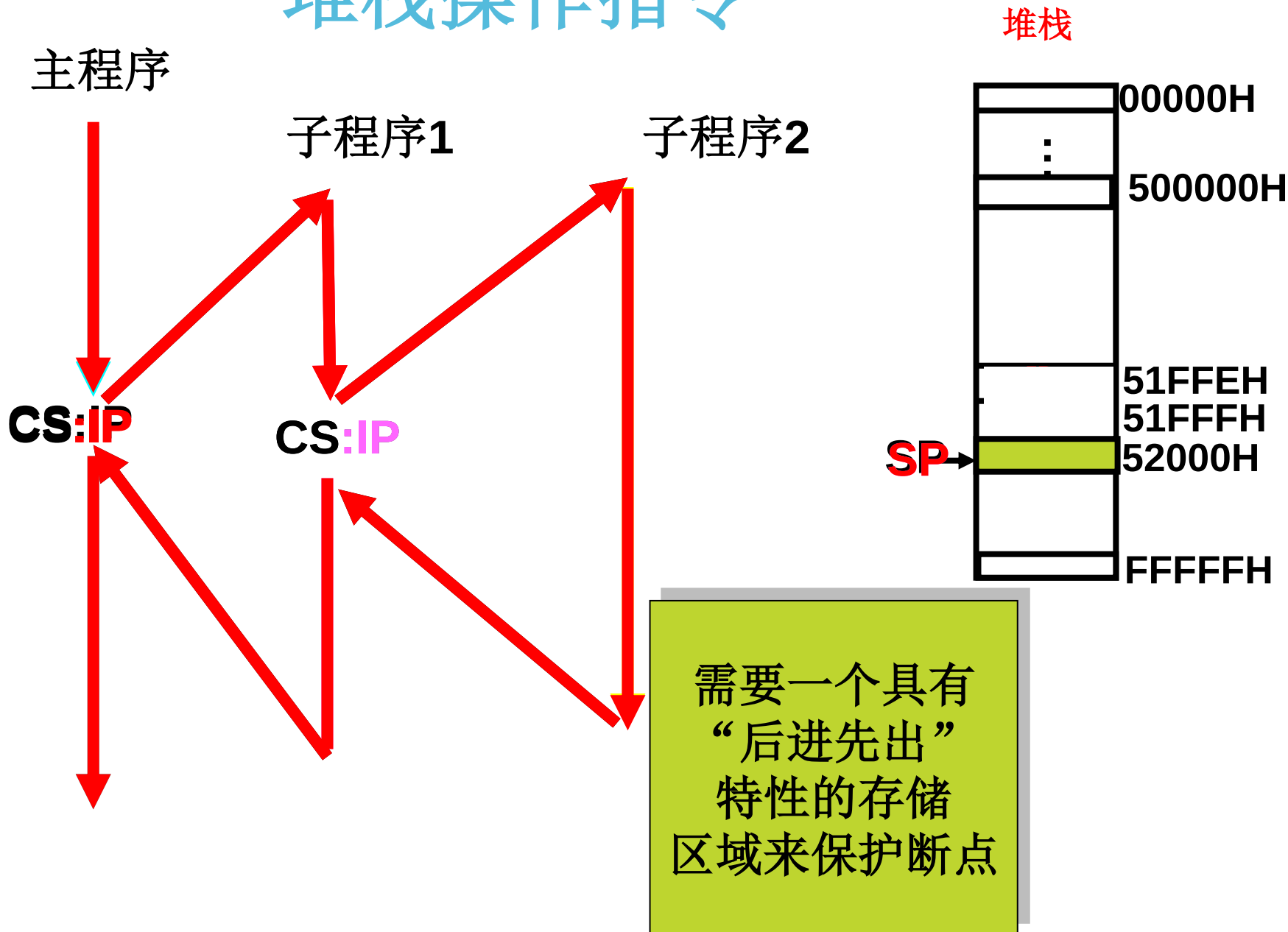
2. 堆栈操作指令

堆栈指令用于中断处理与过程的调用，此时分别要将断点地址和返回地址保存到堆栈区。在进入中断处理和过程后，还需要保留寄存器的值；当中断返回和子程序返回时，则要恢复寄存器的值，这些都要用堆栈指令来完成。

格式：**PUSH Src**
POP Dst

功能：压入堆栈指令PUSH是将16位的源操作数压入堆栈；弹出堆栈指令POP是从栈顶单元弹出16位的源操作数到目标地址。

堆栈操作指令



8086的堆栈操作是按字操作的。每执行一条**PUSH指令**，先将堆栈指针**SP自动减2**，然后将**16位(字)操作数压入栈顶单元**。其**低8位数据存低字节地址单元**，**高8位数据存高字节地址单元**。执行弹出**POP指令**时，操作正好相反，先弹出一个字数据至目标地址，再修正堆栈指针，SP自动加2。

若子程序开始有如下保护现场指令：

```
PUSH AX
```

```
PUSH DS
```

子程序

则子程序返回前，应按如下顺序恢复现场：

```
POP DS
```

```
POP AX
```

(1) 进栈指令PUSH

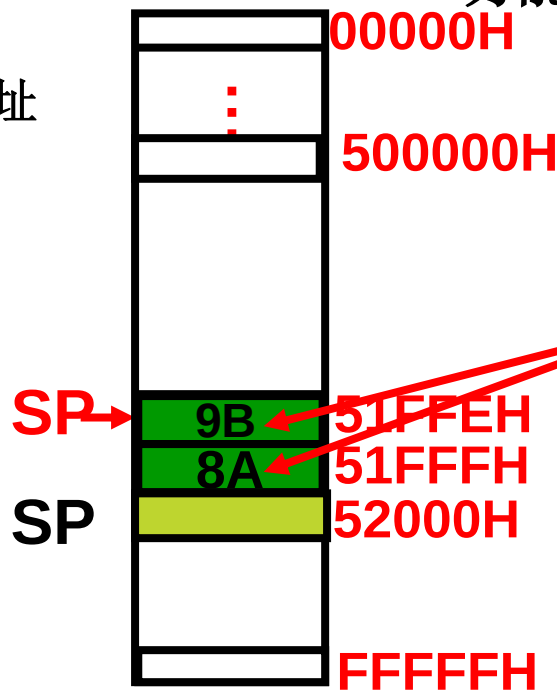
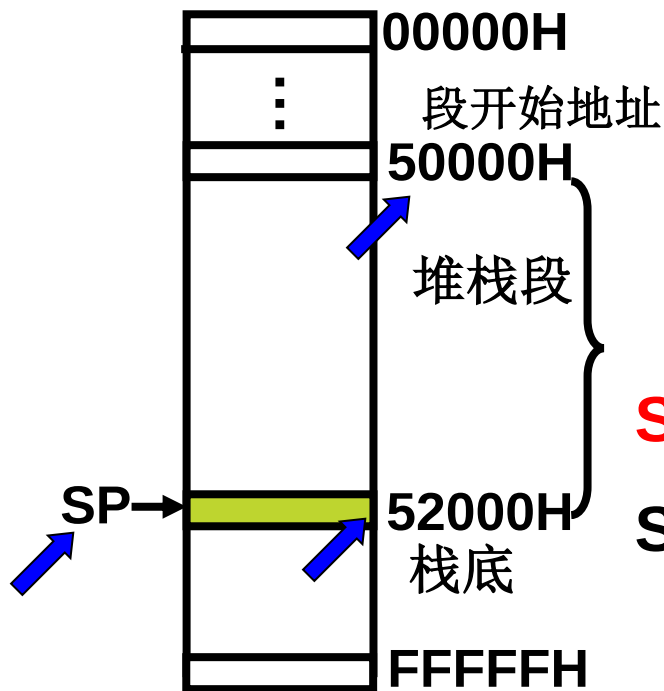
假设: $SS=5000H$, $SP=2000H$

格式: **PUSH OPRD**

功能: $SP-2$, 把源操作数
OPRD 的16位数
据压入堆栈

例: $AX=8A9BH$

执行 **PUSH AX**



• SP自动减2

$(SP+1) \leftarrow (OPRD \text{高4位})$

$(SP) \leftarrow (OPRD \text{低4位})$

• 将AX的内容传送到

执行**PUSH AX**后 **SP**指向的单元

(2) 出栈指令POP

格式: **POP OPRD**

功能: 从堆栈中弹出**16位**数据到目的操作数,**SP**自动加2

(SP) → (OPRD低4位)

(SP+1) → (OPRD高4位)

SP自动加2

将栈区的内容传送OPRD

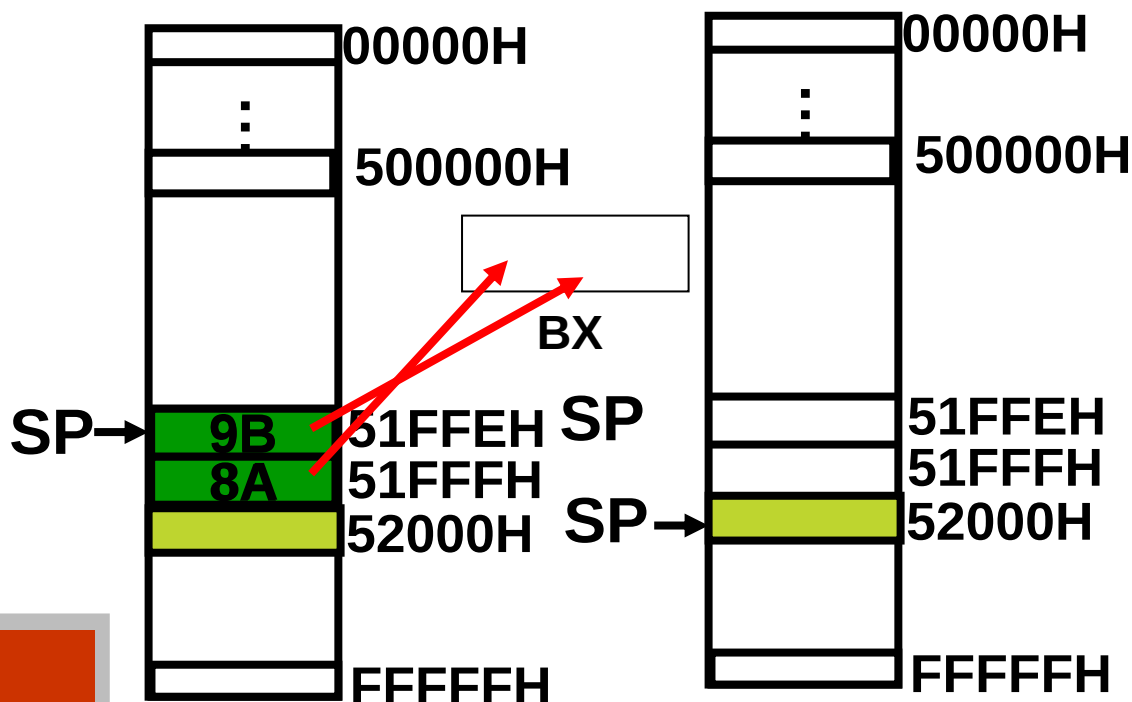
SP指向新栈顶的单元

执行指令 **POP BX**

执行后**BX=8A9BH**

SP自动加2, 指向新的栈顶

可以是通用寄存器、段寄存器
(CS除外) 和字存储单元



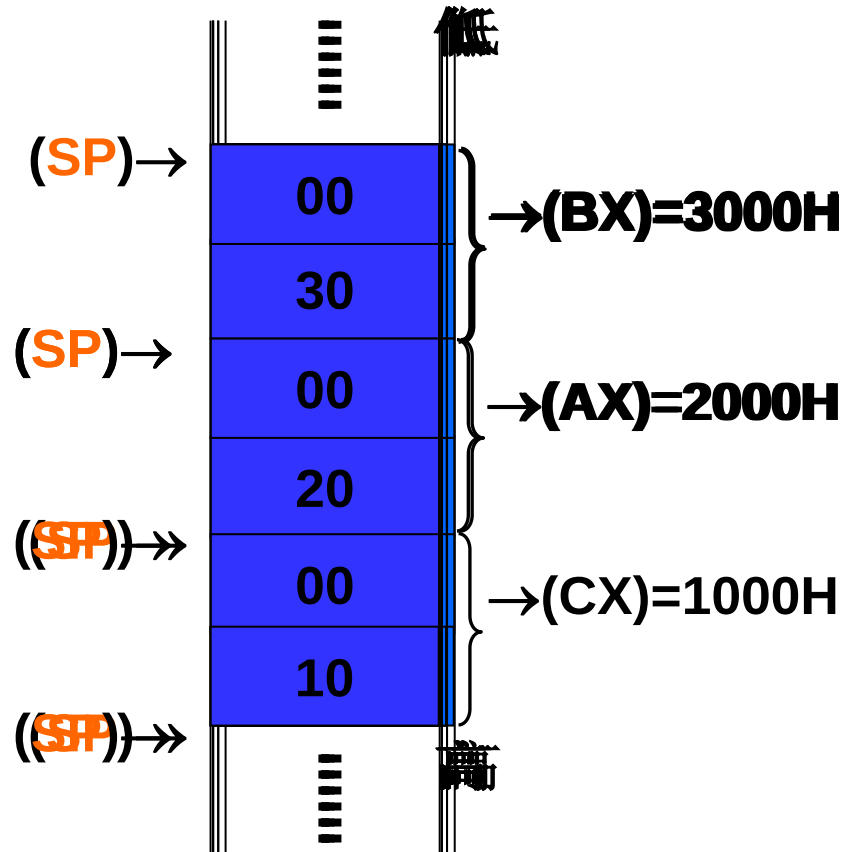
执行POP BX前

执行POP BX后

如下指令表明压栈的源操作数可以是寄存器或内存单元的数，出栈的目的操作数可以是寄存器或内存单元。

PUSH [BX+DI]	;	PUSH MEM
PUSH AX	;	PUSH REG
PUSH DS	;	PUSH SEG
POP [SI]	;	POP MEM
POP AX	;	POP REG
POP ES	;	POP SEG

```
MOV AX , 1000H
MOV BX , 2000H
MOV CX , 3000H
PUSH AX
PUSH BX
PUSH CX
POP  BX
POP  AX
POP  CX
```



3. 交换指令XCHG

格式: XCHG OPRD1, OPRD2

功能: 操作数OPRD1的内容与操作数OPRD2的内容交换

OPRD1和OPRD2可以是通用寄存器和存储单元（两个操作数不能同时为存储单元）可以采用除立即数以外的各种寻址方式

SI

1234H

BX

5678H

1234H

缓存器

例: XCHG AL, AH

XCHG SI, BX

XCHG AL, [SI+3]

XCHG [DI+BX+3], DX

4. 换码指令XLAT

字节转换指令

格式：

XLAT

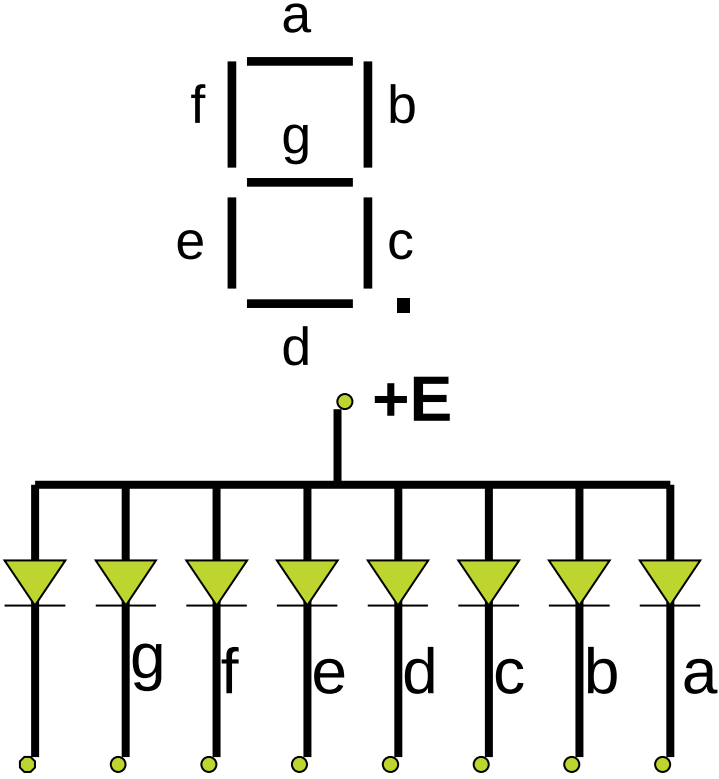
功能：用来将一种字节代码转换成另一种字节代码。它将BX的内容(代码表格首地址)和AL的内容(表格偏移量)相加作为有效地址，并从中取出此地址单元中的内容送到AL中。

XLAT指令可用来将一个代码值转换成相应的另一种代码值。应用时，首先应列出两种代码的表格。

➤例如：要将十进制数0~9转换成共阳极LED显示的字形代码，则列表如**表3-4**所示。

表: BCD码与字形码的相互转换表(共阳极LED)

十进制	BCD 码	字形代码
0	BCDT: 0000	WDT: 40H
1	BCDT+1: 0001	WDT+1: 79H
2	0010	24H
3	0011	30H
4	0100	19H
5	0101	12H
6	0110	02H
7	0111	78H
8	1000	00H
9	1001	18H



换码指令XLATDE的应用

格式: XLAT

功能: 完成一个字节的查表转换。

把累加器AL中的(被查)值, 转换成以BX为表基址的表中值

七段数码管

例: 假设七段码存放在
2000H开始的内存中,
取出3所对应的七段码
(程序片段如下)

```
MOV BX, 2000H
MOV AL, 3 ;
XLAT ;
执行后AL=30H
```

$BX=2000H$
 $AL=3H$
 $2000H$

2009H	10H	9
	00H	.
	78H	.
	02H	.
	12H	5
	19H	4
	30H	3
	24H	2
	79H	1
	40H	0

一个字节的
查表地址

$BX+AL$

...

$BX+AL=2003H$

$BX+2$

$BX+1$

表基址 $BX+0$

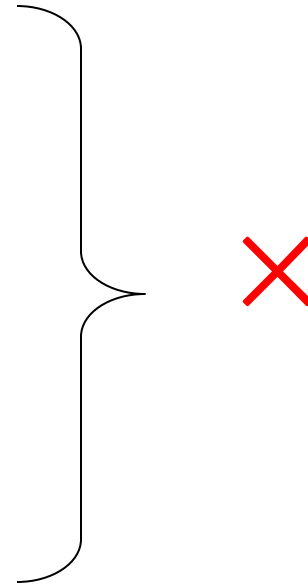
表的段地
址在DS

AL

换码指令的功能

判断对错：

```
PUSH  AL
POP   CS
XCHG  DS, BX
XCHG  [BX], [SI]
POP   BH
XCHG  AX, 1234H
```



3.2.2 目标地址传送指令（1）

1. 取有效地址指令LEA

16位的通用寄存器

格式：LEA reg, src

功能：将源操作数中的16位偏移地址送到一个16位的通用寄存器中。该指令常用来使一个寄存器作为地址指针。

例：LEA AX, [2728H] ; 执行后, AX=2728H

LEA BX, [BP+SI] ; 指令执行后, BX的内容为
BP+SI的值

LEA AX,[DI+10] ; 若(DI)=0500H该指令执行后,
AX的内容(AX)=050AH。

注意：LEA 和 MOV 的区别

3.2.2 目标地址传送指令（2）

2. 地址指针装入寄存器和DS指令

格式：LDS reg , src

功能：把源操作数有效地址指出的双字长的内存单元中的高字内容送入DS，低字内容送入指令中所指定的寄存器中。

例：

指令执行前DS内容为1000H，内存12130H单元内容为78H，12131H单元内容为56H，12132H单元内容为34H，12133H单元内容为12H：

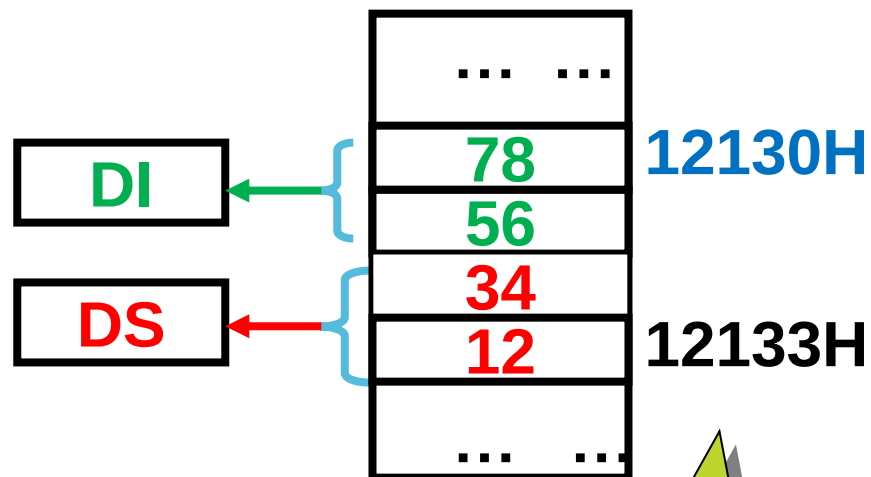
LDS DI, [2130H]

执行此指令后，

将1213**0**H和1213**1**H中的内容(偏移量)送到DI中，DI=5678H；
而将1213**2**H和1213**3**H中内容送到DS中，DS=1234H。

LDS DI, [2130H]

执行后 DS=1234H
DI=5678H



逻辑地址

3. 地址指针装入寄存器和ES指令

格式：LES reg , src

功能：把源操作数有效地址所对应的双字长的内存单元中高字内容(一般为16位段地址)送入ES，低字内容(一般为偏移地址)送入指令所指定的寄存器中。

LES与LDS的指令格式及使用方法相似的。

例如：

LES DI,[2130H]

执行此指令后，将DS段偏移地址2130H和2131H中的内容(**偏移量**)送到DI中，而将DS段偏移地址2132H和2133H中内容(**段值**)送到ES中。

3.2.3 标志位传送指令（1）

1. 读取标志指令LAHF（Load AH with Flags）

格式：LAHF

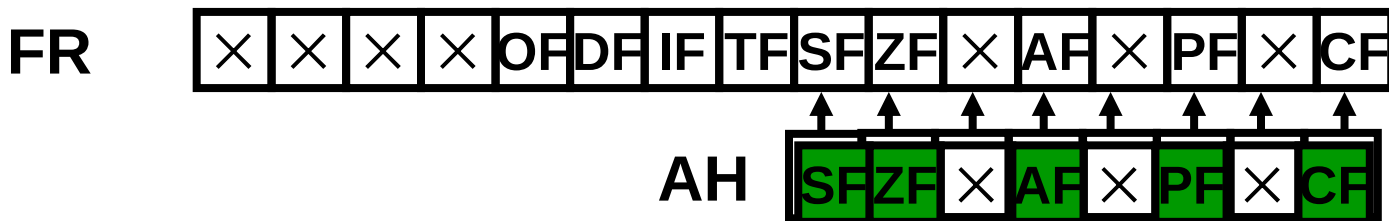
功能：把标志寄存器的低8位对应的传送到寄存器AH



2. 设置标志指令SAHF(Store AH into Flags)

格式：SAHF

功能：把寄存器AH的内容对应的送至标志寄存器的低8位

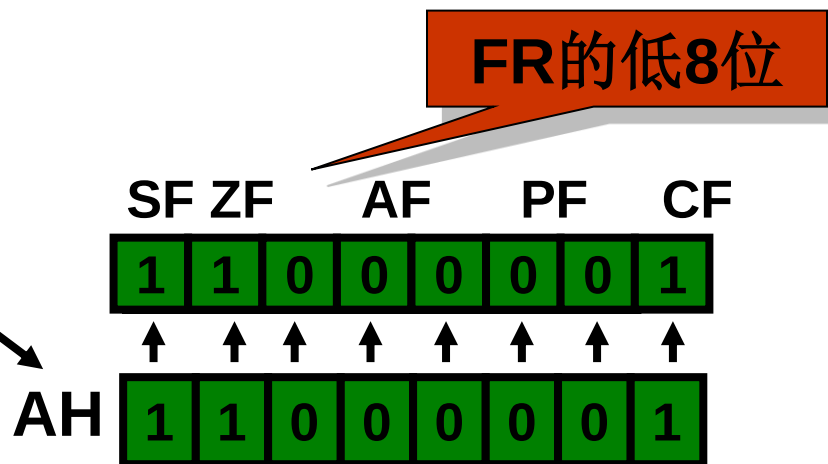


3.2.3 标志位传送指令（2）

例：

MOV AH, 0C1H

SAHF



3. 标志寄存器入栈指令PUSHF

格式：PUSHF

功能：SP-2，把标志寄存器的内容（16位）压入堆栈

4. 标志寄存器出栈指令POPF

格式：POPF

功能：把当前栈顶的一个字传送到标志寄存器，SP+2

3.2.4 输入/输出数据传送指令(1)

1、 输入指令 IN

端口地址可以直接给出
或由DX寄存器间接给出

格式: **IN 累加器, 端口地址**

功能: 从一个端口读取一个字节或一个字, 传送到AL或AX

注意: 外部设备最多可有65536个I/O端口(0000~FFFFH), 只有前256个端口可在指令中直接给出(00~FFH)

例: **IN AL, 50H** ; 将50H端口的字节读入AL

IN AX, 70H ; 将70H、71H两端口的值读入AX

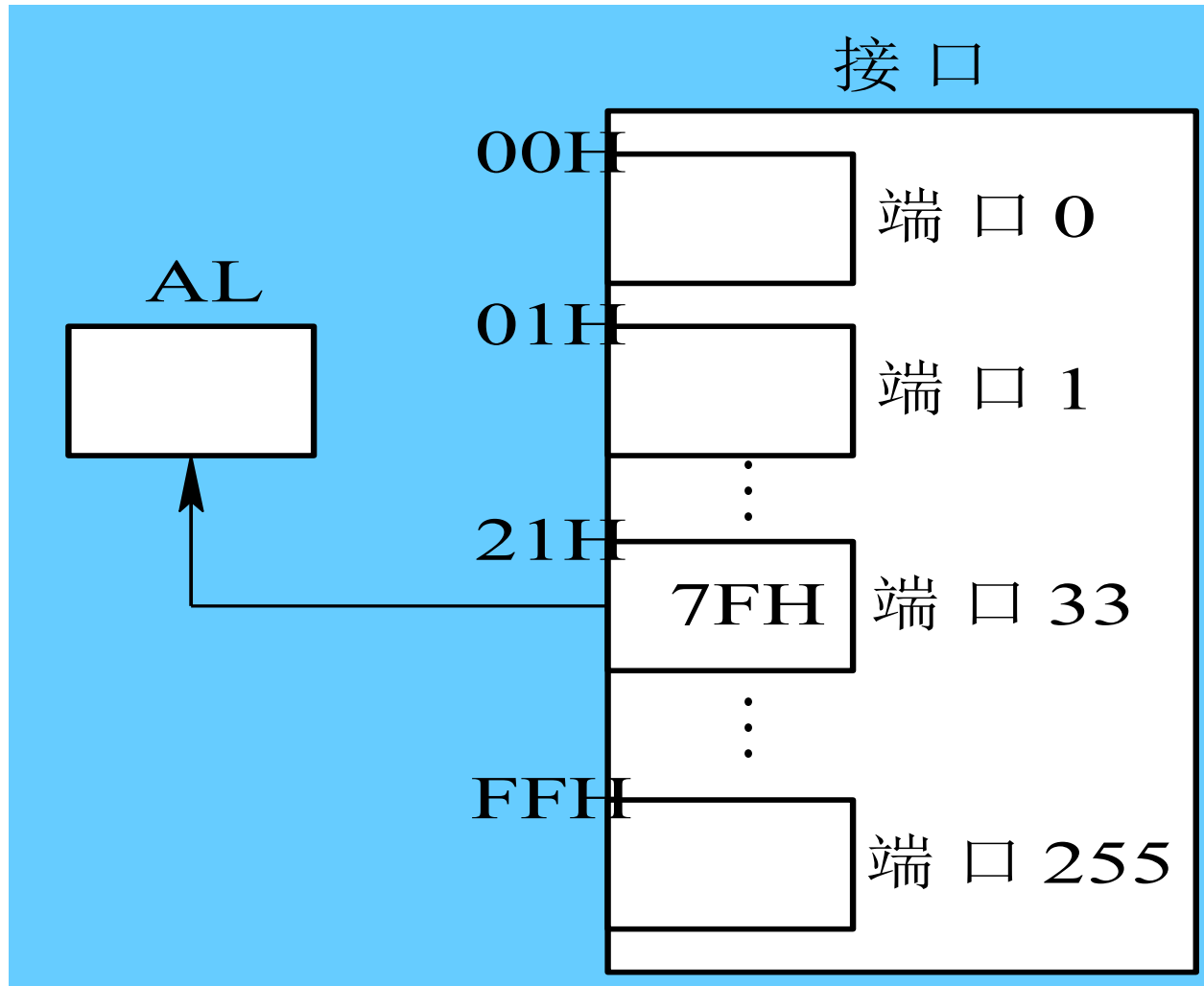
IN AL, DX ; 从DX所指的端口中读取一个字节

IN AX, DX ; 从DX和DX+1所指的两个端口读取一个字

直接寻址
或长格式

间接寻址或短格式

IN AL, 21H



3.2.4 输入/输出数据传送指令（2）

2、 输出指令OUT

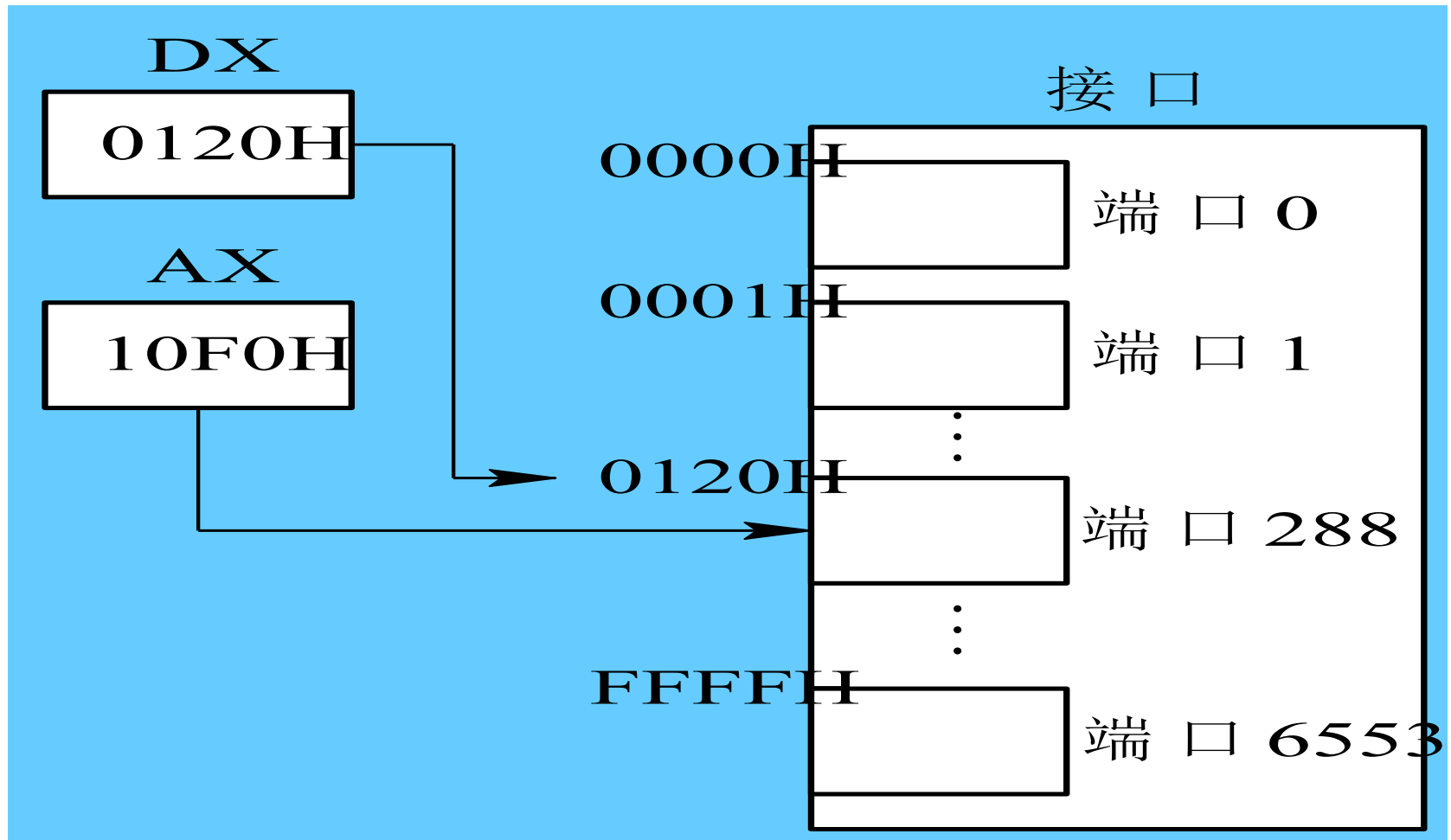
格式： **OUT** 端口地址，累加器

功能： 将**AL**中的一个字节或**AX**中的一个字输出到指定端口

例： **OUT** 70H , AL
 OUT DX , AL

传送类指令不影响标志位

例: **MOV DX , 0120H**
 OUT DX , AL



判断对错:

OUT	258,	AL	}	✗
OUT	DX,	BL		
IN	AL,	1234H		
IN	AL,	BH		
OUT	120,	AL	}	✓
OUT	DX,	AL		
IN	AL,	80H		
IN	AL,	DX		

3.3.1 算术运算类指令

1. 加法指令

(1) 不带进位位的加法指令ADD

格式: **ADD OPRD1, OPRD2 ; (dst)←(dst)+(src)**

功能: 两个操作数相加, 结果送至目的操作数**OPRD1**

例: **ADD AL, 50H ; AL的内容和50H相加, 结果AL中**
ADD CX, 1000H ; CX的内容和1000H相加, 结果放在CX中
ADD DI, SI ; DI和SI的内容相加, 结果放在DI中
ADD [BX+DI], AX ; BX+DI和BX+DI+1两个存储单元和AX
相加, 结果放在BX+DI和BX+DI+1所
指的存储单元中
ADD AX, [BX+2000H]; BX+2000H和BX+2001H所指的两个单元
的内容和AX的内容相加, 结果在AX中

1. 加法指令

(2) 带进位位的加法指令ADC

例：将两个四字节的无符号数相加，两个数分别放在2000H、3000H开始的单元，要求结果放在2000H开始的单元

格式： **ADC OPRD1, OPRD2**; $(dst) \leftarrow (dst) + (src) + (CF)$

功能：两个操作数相加再加上进位标志**CF**，结果送至目的操作数**OPRD1**

```
MOV SI, 2000H ;  
MOV DI, 3000H ;  
MOV AX, [SI] ;  
ADD AX, [DI] ;  
MOV [SI], AX ;  
MOV AX, [SI+2] ;  
ADC AX, [DI+2] ;  
MOV [SI+2], AX ;
```

```
MOV DI,2000H  
MOV SI,3000H  
MOV AX,[SI]  
ADD [DI],AX  
MOV AX,[SI+2]  
ADC [DI+2],AX
```

```
MOV DI,2000H  
MOV SI,3000H  
MOV AX,[SI]  
ADD [DI],AX  
PUSHF  
ADD SI,2  
ADD DI,2  
POPF  
MOV AX,[SI]  
ADC [DI],AX
```

1. 加法指令

(3) 增量指令 INC

通用寄存器、
存储器

格式: **INC OPRD** ; $(dst) \leftarrow (dst) + 1$

功能: 操作数OPRD的内容加1, 结果送回OPRD

该指令结果不影响进位标志CF, 影响其它的五个状态标志

例: **INC AL** ; AL的内容加1
INC CX ; CX的内容加1

INC BYTE PTR[BX][SI] ; 字节存储单元的内容加1
INC WORD PTR[DI] ; 字存储单元的内容加1

2. 减法指令（1）

(1) 不带借位的减法指令SUB

格式: **SUB OPRD1, OPRD2** ; $(dst) \leftarrow (dst) - (src)$

功能: 目的操作数减去源操作数, 结果送到目的标操作数, 源操作数不变

执行结果影响标志寄存器

(2) 带借位的减法指令SBB

格式: **SBB OPRD1, OPRD2**; $(dst) \leftarrow (dst) - (src) - (CF)$

功能: 操作数**OPRD1**减去操作数**OPRD2**的同时还要减去借位（进位）标志**CF**的现行值

执行结果影响标志寄存器

2. 减法指令（2）

(3) 减量指令DEC

格式: **DEC OPRD**; $(dst) \leftarrow (dst) - 1$

功能: 操作数**OPRD**的内容减1, 并把结果送回**OPRD**

执行结果不影响CF, 但影响其它五个状态标志

(4) 取补指令NEG

格式: **NEG OPRD**; $(dst) \leftarrow 0 - (dst)$

实际上是用零
减去操作数

功能: 对操作数取补码, 将结果送回操作数**OPRD**中

执行结果影响状态标志

2. 减法指令（3）

(5) 比较指令CMP

格式： **CMP OPRD1, OPRD2** ； (dst) – (src)

功能： 操作数OPRD1减去OPRD2，运算结果不送到OPRD1但影响状态标志

注意：两个操作数本身都没有发生变化

例如：

CMP AL, 0AH	； 寄存器与立即数比较
CMP CX, DI	； 寄存器与寄存器比较
CMP AX, AREA1	； 寄存器与存储器比较
CMP [BX+5], SI	； 存储器与寄存器比较

总结: 加法指令

格式: `ADD dst, src`

`ADC dst, src`

功能: `ADD`是将`src`与`dst`相加, 结果传送到`DST`。

`ADC`是将`src`与`dst`以及`CF`值相加, 结果传送到`dst`。

其中`src`可以是通用寄存器、存储器操作数或立即数。`dst`可以是通用寄存器或存储器操作数。

➤影响的标志位有`OF`、`SF`、`ZF`、`AF`、`PF`、`CF`。

例如:

`MOV AX, 9876H` ; `AX=9876H`

`ADD AH, AL`

; `AX=0E76H CF=1 SF=0 OF=0 ZF=0 AF=0 PF=0`

`ADC AH, AL`

; `AX=8576H CF=0 SF=1 OF=1 ZF=0 AF=1 PF=0`

总结：减法指令

格式： SUB dst, src

SBB dst, src

功能： SUB将目的操作数减源操作数的结果送dst。

SBB将dst-src-CF(进位/借位)的值送dst。

其中，src可以是通用寄存器、存储器操作数或立即数。dst可以是以寄存器或存储器操作数。

➤ 影响标志位为OF、SF、ZF、AF、PF、CF。

例如：

MOV AX, 9966H ; AX=9966H

SUB AL, 80H

; AL=E6H CF=1 SF=1 OF=1 ZF=0 AF=0 PF=0

SBB AH, 80H

; AH=18H CF=0 SF=0 OF=0 ZF=0 AF=0 PF=1

总结：加1、减1指令

格式： INC dst
DEC dst

功能：INC指令将目的操作数加1，结果送dst。DEC指令将目的操作数减1，结果送dst。其中dst为通用寄存器或存储器操作数。

INC、dst影响标志位为OF、SF、ZF、AF、PF，CF不受影响。

例如：

INC BL	； BL $\leftarrow$ (BL)+1
INC AX	； AX $\leftarrow$ (AX)+1
DEC BYTE PTR[SI]	； 存储器字节操作数减1
DEC CX	； CX $\leftarrow$ (CX)-1

总结:求补指令

格式: NEG dst

功能: 对目的操作数求补, 用零减去目的操作数, 结果送目的操作数单元。其中目的操作数为通用寄存器、存储器操作数。

NEG指令影响标志位OF、SF、ZF、AF、PF、CF。

例如:

(AL)=00010011 ;则执行指令NEG AL的结果是

(AL)=11101101B。

总结:比较指令

格式: CMP dst , src

功能: dst-src, 结果不回送。src为寄存器、存储器操作数或立即数。dst为寄存器、存储器操作数。CMP指令影响标志位为OF、SF、ZF、AF、PF、CF。

例如:

CMP CX , 3

CMP WORD PTR[SI] , AX

CMP AX , BLOCK

3. 乘法指令MUL (IMUL)

有符号数乘法指令

格式: **MUL src** ; **(AX) ← (AL) × (src)** (字节乘法)
; **(DX:AX) ← (AX) × (src)** (字乘法)

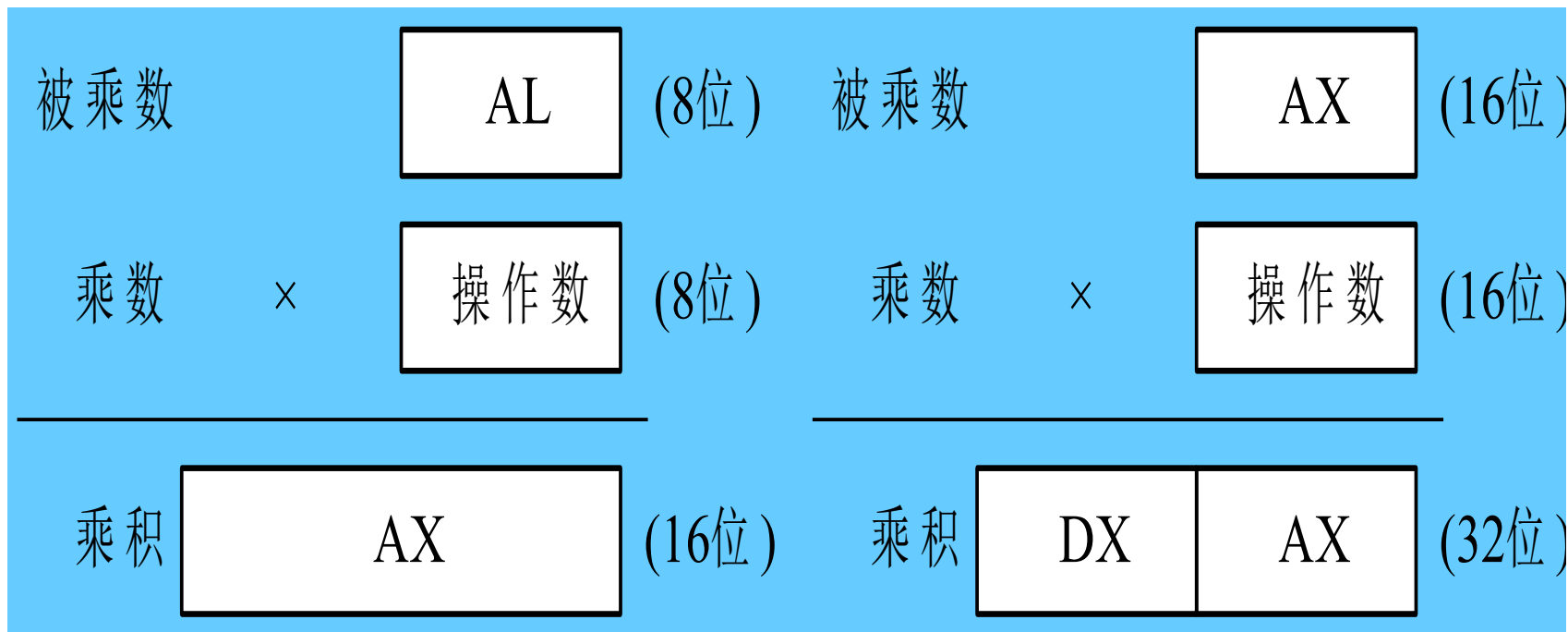
功能: 如果**OPRD**是字节操作数, 则**AL**的内容与**OPRD**相乘

16位结果送到**AX**中。

如果**OPRD**是字操作数, 则**AX**的内容与**OPRD**相乘**32位**的结果送到**DX**和**AX**中, **DX**中是高**16位**, **AX**中是低**16位**

如果乘积结果的高半部分 (字节相乘时为**AH**, 字相乘时为**DX**) 不等于零, 则标志**CF=1**, **OF=1**; 否则**CF=0**, **OF=0**, 对其它标志, 该指令无定义

所以, 如果**CF=0**, **OF=0**, 表示**AH**或**DX**中的结果数无效。



MUL AL ; AL乘以AL, 结果在AX中
MUL BX ; AX乘以BX, 结果在DX:AX中
MUL BYTE PTR[DI+6] ; AL乘以存储器(8位), 结果在AX中
MUL WORD PTR ALPHA ; AX乘以存储器(16位), 结果在DX:AX中

例：假设 (AL) = 0B4H, (BL) = 11H

①求执行MUL BL的乘积值 ②求执行IMUL BL 的乘积值

无符号数

0B4H = 10110100B
取补码 = 4CH
= -76D

90D
D

有符号数：

(AL) = 0B4H = -76D

(BL) = 11H = 17D

-76D

× 17D

-1292D

-1292D = -10100001100B
= 0FAF4H (补码)

-1292D = 0FAF4H

执行 IMUL BL 后

(AX) = 0FAF4H

(AH) 不是符号扩展 ∴ CF=OF=1

AH ≠ 0 ∴ CF=OF=1

4. 除法指令DIV (IDIV)

有符号数除法指令

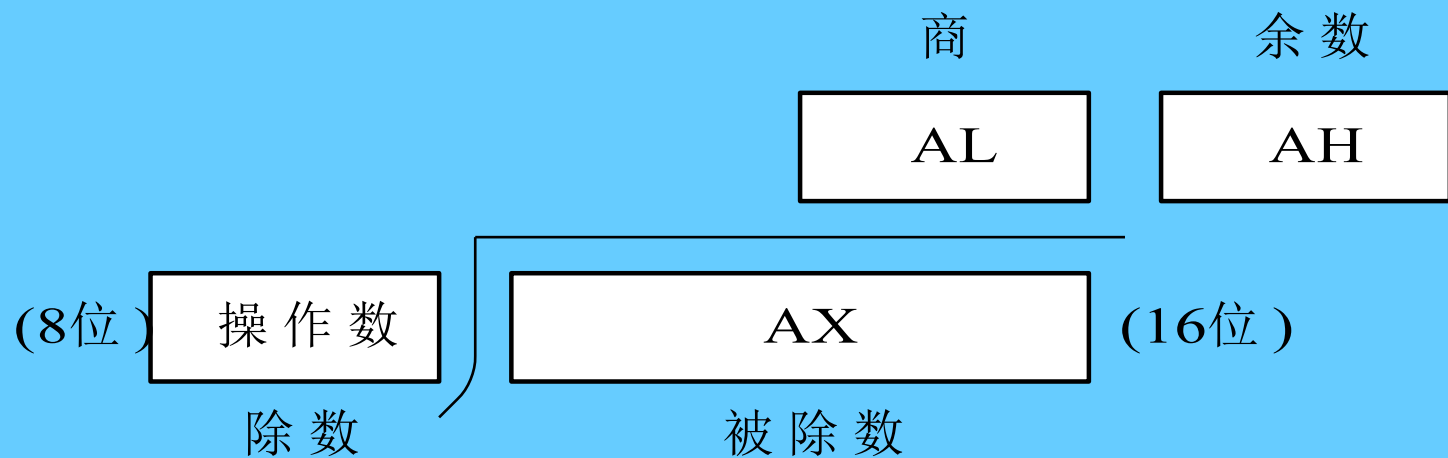
格式: **DIV src** ; (AL) $\leftarrow$ (AX)/(src)的商 (字节除法)
; (AH) $\leftarrow$ (AX)/(src)的余数
; (AX) $\leftarrow$ (DX:AX)/(src)的商 (字除法)
; (DX) $\leftarrow$ (DX:AX)/(src)的余数

功能: 如果src是字节操作数, 则AX的内容除以src
8位的商送到AL中。8位余数送到AH中

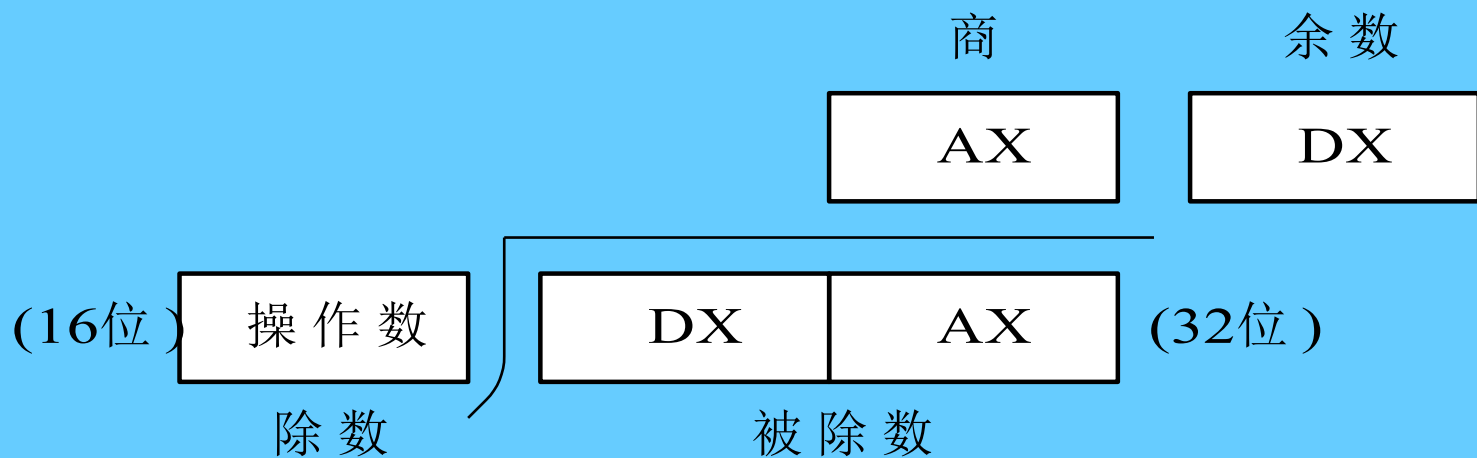
如果src是字操作数, 则DX中(高16位)和AX中
(低16位)的内容除以src, 16位的商送到AX中。
16位的余数送到DX中。

除法指令对标志位的影响无定义

注意: 如果除数为0, 或者8位数除时商超过8位, 或者
16位除时商超过16位, 则认为是溢出, 引起0号中断



(a)



(b)

在DIV指令中，一个操作数(被除数)隐含在累加器AX(字节除)或DX:AX(字除法)中，另一个操作数src(除数)必须是寄存器或存储器操作数。两个操作数被作为无符号数对待。

例如：

DIV BL ; AX除以BL

DIV CX ; DX:AX 除以CX

DIV BYTE PTR DATA ; AX除以存储器(8位)

DIV WORD PTR[DI+BX] ; DX:AX除以存储器(16位)

(1) 字节转换为字命令CBW (Convert Byte to Word)

格式: CBW ; 如果(AL)<80H, 则(AH)←00H, 否则(AH)←0FFH

功能：寄存器AL中的符号扩展到寄存器AH。若AL的最高有效位为0，则AH=0，否则AH=0FFH;对状态标志位没有影响。

(a) MOV AL, 4FH ; (AL)=01001111B
CBW ; (AH)=00000000B

(b) MOV AL, 0F4H ; (AL)=11110100B
CBW ; (AH)=11111111B

(2) 字转换为双字指令CWD (Convert Word to Double word)

格式: **CWD** ; 如果(AX)<8000H, 则(DX)←0000H, 否则(DX)←FFFFH

功能：寄存器AX中的符号扩展到寄存器DX。若AX的最高有效位为0，则DX=0，否则DX=0FFFFH；对状态标志位没有影响。

例：计算表达式的值： $(X*Y+Z-1024)/75$

假设其中的X、Y、Z均为16位带符号数，分别放在名为XXX、YYY、ZZZ的变量单元中。

```
...  
MOV     AX, XXX  
IMUL    YYY           ; 计算X*Y(结果在DX, AX中)  
MOV     CX, AX  
MOV     BX, DX       ; 积保存到BX: CX中  
MOV     AX, ZZZ  
CWD                     ; 把AX扩展为32位 (扩展到DX)  
ADD     AX, CX  
ADC     DX, BX  
SUB     AX, 1024      ; 计算差  
SBB     DX, 0         ; 减借位  
MOV     CX, 75  
IDIV    CX           ; 计算商和余数  
...
```

X*Y的值

DX: AX是Z值

结果商保存在AX中，余数保存在DX中

总结:乘法指令

格式: MUL src

IMUL src

功能: MUL为无符号数乘法指令, IMUL为带符号数乘法指令。

其中src为通用寄存器或存储器操作数。目的操作缺省存放在AL或AX中, 乘积存在AX(字节乘)或在DX、AX中(字乘)。

字节乘: $AL * src \rightarrow AX$

字 乘: $AX * src \rightarrow DX:AX$

例如:

MUL BL ; 字节乘

MUL WORD PTR[SI] ; 字乘

IMUL BYTE PTR[DI] ; 字节乘

总结:除法指令和符号扩展指令

格式: **DIV src**

IDIV src

CBW

CWD

功能: DIV为无符号数除法, IDIV为带符号数除法。其中src为除数, 为通用寄存器或存储器操作数。被除数缺省在目的操作数AX、DX或AX中。

字节除法: AX/src8 商→AL, 余数→AH

字除法: DX:AX/src16 商→AX, 余数→DX

字节扩展为字: CBW指令将AL中带符号数的符号位扩展到AH中

字扩展为双字: CWD指令将AX中带符号数的符号位扩展到DX中

由于被除数必须是除数的双倍字长, 带符号数除法可以先使用符号扩展指令进行符号扩展。当进行无符号数除法时, 被除数高位按0扩展为双倍除数长度。当进行带符号数除法时, 被除数以补码表示。可使用扩展指令CBW、CWD进行符号扩展。

例如：

MOV AX,8000H

CWD ；被除数符号扩展,D_X为0FFFFH,A_X为8000H

MOV BX,1000H

IDIV BX

对于带符号数除法，其商和余数均采用补码形式表示，余数与被除数同符号，当除数为零或商超过了规定数据类型所能表示的范围时，将会出现溢出现象，产生一个中断类型码为“0”的中断。执行除法指令后标志位无定义。

	带 符 号 数	无 符 号 数
8位 除以 8位	a.把被除数放入（ AL） b.CBW c.（ AX）除以除数（ IDIV）	a.把被除数放入（ AL） b.（ AH）=0 c.（ AX）除以除数（ DIV）
16位 除以 16位	a.把被除数放入（ AX） b.CWD c.（ DX： AX）除以除数（ IDIV）	a.把被除数放入（ AX） b.（ DX）=0 c.（ DX： AX）除以除数（ DIV）

6. BCD算术运算调整指令

十进制数(BCD码)运算指令

以上我们介绍的是二进制数的算术运算指令，二进制数在计算机上进行运算是非常简单的。但是，通常人们习惯于用十进制数。

- 在计算机中十进制数是用BCD码来表示的，BCD码有两类：一类叫压缩型BCD码，一类叫非压缩型BCD码。
- 用BCD码进行加、减、乘、除运算，通常采用两种方法：一种是在指令系统中设置一套专用于BCD码运算的指令；另一种是利用二进制数的运算指令算出结果，然后再用专门的指令对结果进行修正(调整)，使之转变为正确的BCD码表示的结果。8086/8088指令系统所采用的是后一种方法。
- 在进行十进制数算术运算时，应分两步进行：先按二进制数运算规则进行运算，得到中间结果；再用十进制调整指令对中间结果进行修正，得到正确的结果。

48 + 29 = 77 0100 1000 48的BCD码

+ 0010 1001 29的BCD码

0111 0001 71的BCD码

$$48 + 29 = 77$$

0100 1000

48的BCD码

0100 1000

+ 0010 1001

29的BCD码

+ 0010 1001

0111 0001

71的BCD码

0111 0001

中间结果 AF=1

+ 0000 0110

加06H调整

0111 0111

正确结果, 77的BCD码

加法运算后, 低4位若向高4位有进位(即AF=1), 调整指令应做加06H处理。

$$57 + 46 = 103$$

0101 0111

+ 0100 0110

1001 1101

中间结果

+ 0000 0110

1010 0011

中间结果

+ 0110 0000

CF←1 0000 0011 正确结果 CF=1

十进制数可以采用BCD码表示，以压缩格式存放，即一个字节存储2位BCD码，BCD加减法是在二进制加减运算的基础上，对其二进制结果进行调整，将结果调整成BCD码格式。

(1) DAA

格式：DAA

功能：将存放在AL中的二进制和数，调整为压缩格式的BCD码。

调整方法：若AL中的低4位 >9 或标志AF=1，

则 $AL+6 \rightarrow AL$ ， $1 \rightarrow AF$ 。

若AL中高4位 >9 或标志CF=1，则 $AL+60H \rightarrow AL$ ， $1 \rightarrow CF$ 。

DAA指令一般紧跟在ADD或ADC指令之后，影响标志为SF、ZF、AF、PF、CF，OF无定义。

例：ADD AL,BL

DAA

(2) DAS

格式：DAS

功能：将存放在AL中的二进制差数，调整为压缩的BCD码。

调整方法：若AL中低4位 >9 或标志AF=1，则AL-6 $\rightarrow$ AL，1 $\rightarrow$ AF。若AL中高4位 >9 或标志CF=1，则AL-60H $\rightarrow$ AL，1 $\rightarrow$ CF

DAS一般跟在SUB或SBB之后使用，影响标志位为SF、ZF、AF、PF、CF，OF无定义。

例如：

SUB AL,BL

DAS

7. ASCII算术运算调整指令

数字字符 ‘0’～ ‘9’的ASCII码为30H～39H，采用一字节存放1个ASCII码9. ASCII算术运算调整指令

对于ASCII码的算术运算是在二进制运算基础上进行调整。有加、减、乘、除四种调整指令。

(1) AAA

格式：AAA

功能：将存放在AL中的二进制和数，调整为非压缩的BCD码（一个字节存放一位BCD码）。

调整方法：若AL中低4位小于或等于9，且AF=0,则将AL中高4位清0，AF→CF。若AL中低4位大于9或AF=1，则AL+6→AL，AH+1→AH，1→AF，AF→CF，AL中高4位清0。

AAA指令是跟在ADD或ADC指令之后使用，影响标志位为AF、CF，其他标志位无定义。

例如：6+5=11

MOV AX,0036H ; AH内容为0，AL内容为字符 ‘6’的ASCII码

ADD AL,35H ; AL加字符 ‘5’的ASCII码

AAA ; AX=0101H→11，结果为BCD码11

(2) AAS

格式：AAS

功能：将存放在AL中的二进制差数，调整为非压缩的BCD码。

调整方法：若AL中低四位小于或等于9，且AF=0，则AL高4位清0，AF→CF。若AL中低4位大于9或AF=1，则AL-6→AL。AH-1→AH，1→AF，AF→CF，AL中高4位清0。

➤ AAS一般跟在SUB、SBB指令之后使用，影响标志位为AF、CF，其他标志位无定义。

例如：12-5=7

MOV AX,0132H

SUB AL,35H

AAS ; AX=0007H

(3) AAM

格式：AAM

功能：将存放在AL中的二进制积数，调整为非压缩的BCD码。

调整方法：AL/10 商→AH，余数→AL。

➤ AAM指令一般跟在MUL之后使用，影响标志位为SF、ZF、PF，其他标志位无定义。

例如：

MOV AL,07H

MOV BL,09H

MUL BL ; AX=003FH

AAM ; AX=0603H

(4) AAD

格式：AAD

功能：将AX中两位非压缩的BCD码，转换为二进制数的表示形式。

调整方法： $AH * 10 + AL \rightarrow AL$ $0 \rightarrow AH$

➤ AAD用于DIV操作之前，影响的标志位为SF、ZF、PF，其他标志位无定义。

例如：

MOV AX,0605H

MOV BL,09H

AAD ; AX=0041H

DIV BL ; AX=0207H

3.3.2 逻辑运算和位移指令

1. 逻辑运算指令（AND、OR、NOT、XOR、TEST）

(1) 与操作指令AND

结果影响ZF、SF、PF

格式： **AND OPRD1, OPRD2**

功能：两个操作数进行按位的逻辑“与”运算，结果送到目的操作数**OPRD1**，**CF=0, OF=0**

- 1.可屏蔽某些不关心的位(要保留位和1与,不保留位和0与);
- 2.自身与值不变; 3.清进位**CF**

例1:

```
      1010 1001
AND  0000 1111
-----
      0000 1001
```

例2:

AND AL, AL

CF=0
OF=0

1. 逻辑运算指令

(2) 或操作指令OR

结果影响ZF、SF、PF

格式: **OR OPRD1, OPRD2**

功能: 两个操作数进行按位的逻辑“或”运算, 结果送到目的操作数**OPRD1**, **CF=0, OF=0**

1.可对某些位置**1**(要置位和**1**或, 保留位和**0**或);

2.自身或值不变; 3.清进位**CF**

(3) 非操作指令NOT

对标志没有影响

格式: **NOT OPRD**

功能: 把操作数**OPRD**按位取反, 结果送回**OPRD**

例如:

NOT AX

指令执行前, **(AX)=AAAAH**, 执行后**(AX)=5555H**

1. 逻辑运算指令

(4) 异或操作指令XOR

结果影响ZF、SF、PF

格式: **XOR OPRD1, OPRD2**

功能: 两个操作数进行按位的逻辑“异或”运算, 结果送到目的操作数**OPRD1**, **CF=0, OF=0**

- 1. 可对某些位求反(要求反位和1异或, 保留位和0异或);
- 2. 自身异或值清零; 3. 清进位**CF**

(5) 测试指令TEST

格式: **TEST OPRD1, OPRD2**

功能: 两个操作数进行按位的逻辑“与”运算, 结果**不送**到目的操作数**OPRD1**, 仅仅影响标志位

通常用于检测某位是否为1, 但又不改变原来值的场合

例：

1、检查AL中的第6位或第2位是否有一位为1

TEST AL, 01000100B → TEST AL, 44H

执行指令后，如果6、2位都为0，ZF=1，否则ZF=0

2、要求屏蔽AL寄存器中的第6位和第4位

AND AL, 0AFH → 10101111B

3、要求AL寄存器中的第0位和第1位变反

XOR AL, 03H → 00000011B

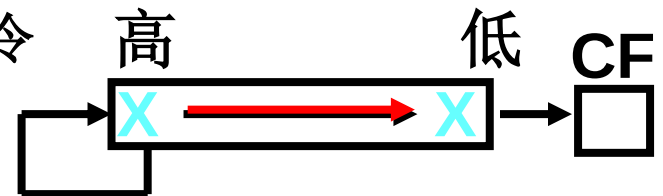
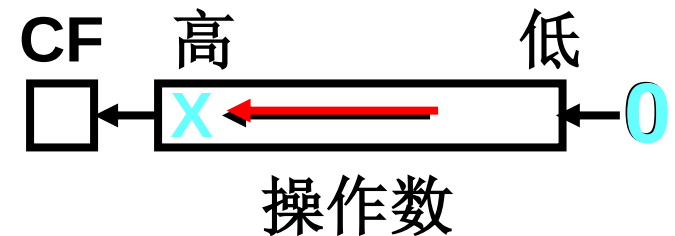
(1) 非循环移位指令 Shift Arithmetic Left

格式: **SAL** OPRD, m ; 算术左移指令
SHL OPRD, m ; 逻辑左移指令
SAR OPRD, m ; 算术右移指令
SHR OPRD, m ; 逻辑右移指令

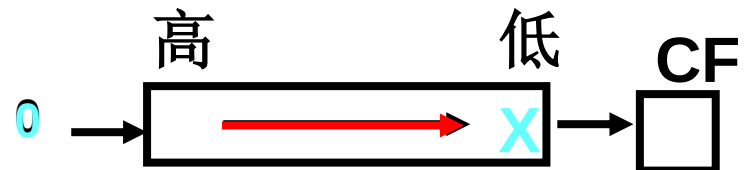
Shift logic Right

功能: 操作数左或右移m位

m为1或为CL
当要移多位时, 移位位数
需存放在CL寄存器中。



算术右移SAR



逻辑右移SHR

(2) 循环移位指令

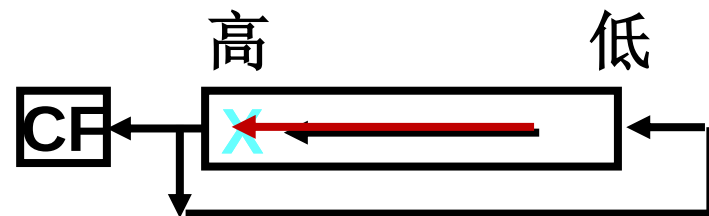
格式:

Rotate Right (left)

ROL OPRD, m ; 循环左移指令
ROR OPRD, m ; 循环右移指令
RCL OPRD, m ; 带进位循环左移指令
RCR OPRD, m ; 带进位循环右移指令

功能: 操作数左或右循环移m位, m是移位位数, 为1或为CL, 当要移多位时, 移位位数需存放在CL寄存器中。

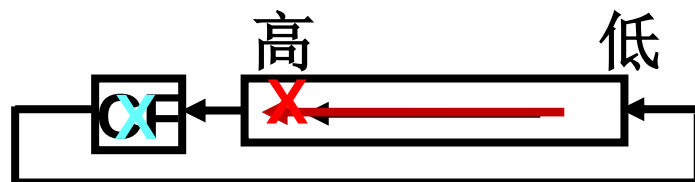
只影响标志CF和OF



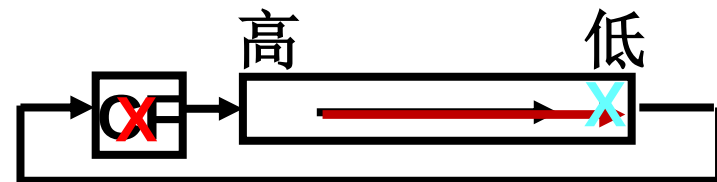
循环左移指令ROL



循环右移指令ROR



带进位循环左移指令RCL



带进位循环右移指令RCR

例：不用乘法指令，用移位指令编程实现 **AL**寄存器的内容乘以**10**（假设结果仍然是**8**位）

SAL **AL** , **1** ; **AL**中的数左移**1**位（**×2**）

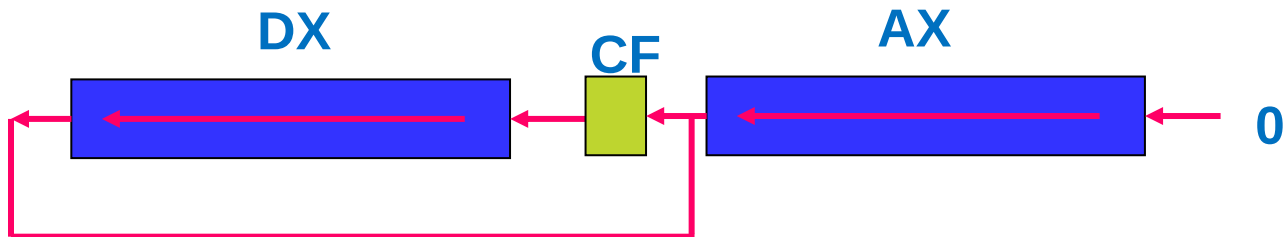
MOV **BL** , **AL**

MOV **CL** , **2**

SAL **AL** , **CL** ; **AL**中的数再左移**2**位（**×8**）

ADD **AL** , **BL** ; **AL**中的数**×10**

例：不用乘法指令，用移位指令编程实现 **DX**和**AX**寄存器的内容乘以**2**（假设结果仍然是**32**位）



SHL **AX** , **1** ; **AX**左移**1**位，**AX**的最高位移到**CF**

RCL **DX** , **1** ; **DX**带进位左移**1**位，**CF**移到**DX**的
最低位，**DX**的最高位移到**CF**中。

例：位传送

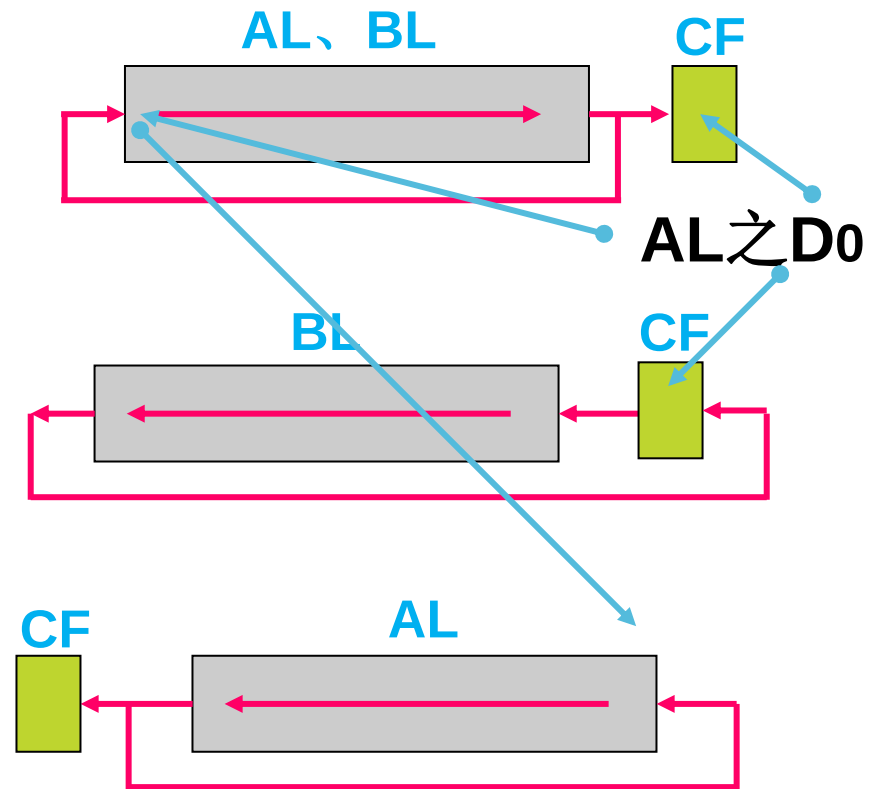
； 把AL最低位送BL最低位， 保持AL不变

ROR BL, 1

ROR AL, 1

RCL BL, 1

ROL AL, 1



例：BCD码合并

； AH.AL分别存放着非压缩BCD码的两位

； 将它们合并成为一个压缩BCD码存AL

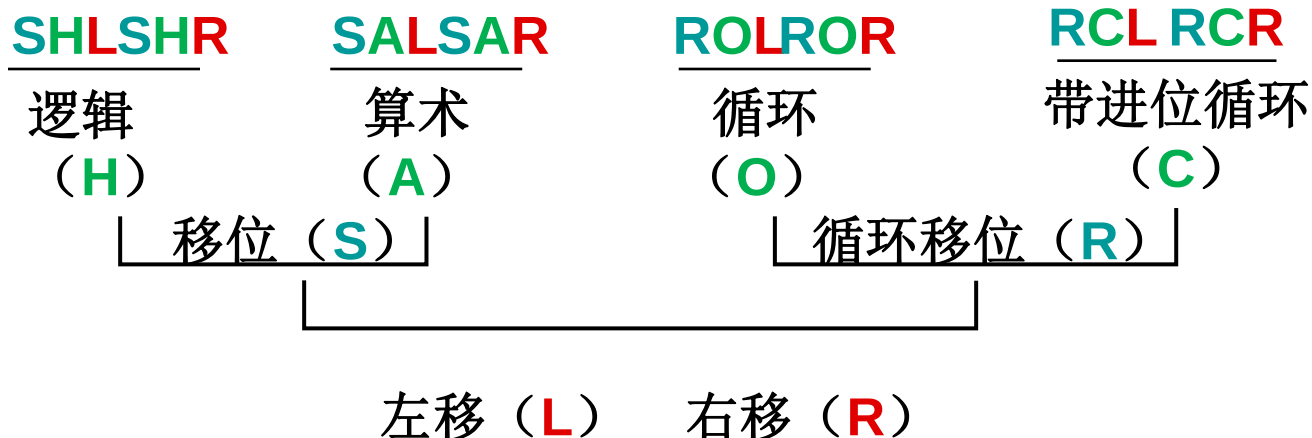
AND AX,0F0FH ； 保证高4位为0

MOV CL,4

ROL AH,CL ； 也可以用SHL AH,CL

ADD AL,AH ； 也可以用OR AL,AH

移位指令小结



- (1) 移位为单线，循环移位为闭环
- (2) 每一种移位都将移出的数位放入CF中
- (3) 逻辑/算术：算术带符号位，保持最高位不变
- (4) 两种循环：循环—CF不参加；带进位循环—CF在环中
- (5) OP OPR, CNT : 循环次数=1时 CNT=1
循环次数>1时 CNT为CL

3.4 串操作类指令

串操作指令是8088指令系统中比较独特的一类指令，采用比较特殊的数据串寻址方式，在操作主存连续区域的数据时，特别好用、因而常用。

重点掌握： **MOVS; STOS; LODS**

CMPS; SCAS; REP

一般了解： **REPZ/REPE REPNZ/REPNE**

3.4 串操作指令

字符串：字符的一个序列

字符串操作的几点说明：

- 由变址寄存器 **SI** 指向 **源操作数**（串）
- 由变址寄存器 **DI** 指向 **目的操作数**（串）
- 规定源串放在当前 **数据段** 中(**DS**)，目的串放在当前 **附加段** 中(**ES**)

即： **DS: SI** 指向源串
 ES: DI 指向目的串

3.4.1 字符串的传送 (MOVS)

格式: **MOVSB** 或 **MOVSW**

注意: 字符串传送指令
不影响标志

功能: **MOVSB**指令把寄存器**DS:SI**所指向的一个字节数据传送到由寄存器**ES:DI**所指向的存储单元中, 然后根据方向标志**DF**确定**SI、DI 增 1 还是减 1**, **DF=1**, **SI、DI**自动减1, 若**DF=0**, **SI、DI**自动加1。

MOVSW指令把寄存器**DS:SI**所指向的一个字数据传送到由寄存器**ES:DI**所指向的存储单元中, 然后根据方向标志**DF**确定**SI、DI 增 2 还是减 2**, **DF=1**, **SI、DI**自动减2, 若**DF=0**, **SI、DI**自动加2。

在执行**MOVS**前, 必须确定方向, 即**DF**置位1或复位0
增址传送时必须确定串首地址, 减址传送时必须确定串尾地址,

串指令使用的一般方法

设置源串地址

MOV SI, 源串首地址
(或LEA SI, 源串)

设置目标串地址

MOV DI, 目的串首地址
(或LEA DI, 目的串)

设置串长度

MOV CX, 串长度

设置操作方向DF

CLD (或STD)

串指令

《串指令》

例：用串传送指令把**1000H**单元的**3**个字节的数据
传送到**2000H**单元

CLD

```
MOV    SI , 1000H
MOV    DI , 2000H
MOVSB
MOVSB
MOVSB
HLT
```

LP:

CLD

```
MOV    SI , 1000H
MOV    DI , 2000H
MOV    CL , 03
MOVSB
DEC    CL
JNZ    LP
HLT
```

CLD

```
MOV    SI , 1000H
MOV    DI , 2000H
MOV    CX , 03H
REP  MOVSB
HLT
```

3.4.2 字符串的存和取（1）

1. 字符串存指令（STOS）

注意：字符串存指令
不影响标志

格式：STOSB 或 STOSW

功能：把累加器的值存到内存中，如果使用了重复前缀
REP，即对某个区域填充累加器的内容。

STOSB（字节存）

把累加器AL的内容送到寄存器ES:DI所指向的存储单元中，然后根据方向标志DF确定DI增1还是减1，DF=1，DI自动减1，若DF=0，DI自动加1。

STOSW（字存）

把累加器AX的内容送到寄存器ES:DI所指向的存储单元中，
然后根据方向标志DF确定DI增2还是减2，DF=1，DI自动减2，
若DF=0，DI自动加2。

程序举例：将当前段1000H开始的64个单元清零

CLD ; 清方向标志DF

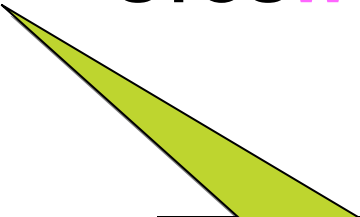
LEA DI, [1000H]

MOV CX, 20H ; 传送32个数据

XOR AX, AX

REP STOSW ; 将32个字清零

HLT



重复CX次

3.4.2 字符串的存和取（2）

2. 字符串取指令（LODS）

格式：LODSB 或LODSW

功能：把内存中的值取到累加器

注意：字符串取指令
不影响标志

LODSB（取字节）

把寄存器DS:SI所指向存储器的一个字节数据取到累加器AL中，然后根据方向标志DF确定SI增1还是减1，DF=1，SI自动减1，若DF=0，SI自动加1。

LODSW（取字）

把寄存器DS:SI所指向存储器的一个字数据取到累加器AX中，然后根据方向标志DF确定SI增2还是减2，DF=1，SI自动减2，若DF=0，SI自动加2。

3.4.3 字符串的扫描和比较 (1)

1. 字符串扫描指令 (SCAS)

格式: SCASB 或 SCASW

功能:

SCASB (字符串字节扫描)

把累加器AL的内容与寄存器ES:DI所指向的一个字节数据采用相减方式比较, 结果影响各标志位, 不影响两个操作数, 若相同, 则ZF=1。根据方向标志DF(0/1), DI增减1。

SCASW (字符串字扫描)

把累加器AX的内容与寄存器ES:DI所指向的一个字数据采用相减方式比较, 结果影响各标志位, 不影响两个操作数, 若相同, 则ZF=1。根据方向标志DF(0/1), DI增/减2。

例： 在某字符串中查找是否存在“\$”字符（该字符串的首地址为1000H，长度为32个字节），如果存在把所在地址送入BX寄存器中，否则BX寄存器清0。

CLD

MOV DI, 1000H

MOV CX, 0020H

MOV AL, '\$'

REPNE SCASB

； AL的内容与串中字符比较，
结果不为零重复

AND CX, 0FFFFH

JZ ZER

DEC DI

MOV BX, DI

JMP ST0

通过CX值判断是否找到“\$”，“与”的结果为“0”，没找到，否则，找到了，地址为DI-1。

ZER: MOV BX, 0

ST0: HLT

3.4.3 字符串的扫描和比较 (2)

1. 字符串比较指令 (CMPS)

格式: **CMPSB** 或 **CMPSW**

功能:

CMPSB (字符串字节比较)

把寄存器 **DS:SI** 所指向的一个字节的数据与寄存器 **ES:DI** 所指向的一个字节的数据采用相减方式比较, 相减结果影响各有关标志位, 不影响两个操作数, 若相同, 则 **ZF=1**。根据方向标志 **DF(0/1)**, 确定 **SI** 和 **DI** 增1或减1

CMPSW (字符串字比较)

把寄存器 **DS:SI** 所指向的一个字的数据与寄存器 **ES:DI** 所指向一个字的数据采用相减方式比较, 相减结果影响各有关标志位, 不影响两个操作数, 若相同, 则 **ZF=1**。根据方向标志 **DF(0/1)**, 确定 **SI** 和 **DI** 增2或减2

3.4.4 重复前缀REP

REP 后面一般跟
MOVS或STOS

格式: REP 串操作指令

功能: 重复后面的串操作指令, (CX) 次

REP ; 每执行一次串指令, CX减1
; 直到CX=0, 重复执行结束

例: 要求把当前数据段中偏移1000H开始的100个字节的数据传送到2000H开始的单元中。

CLD	; 清方向标志 (DF=0, 按增值方式调整指针)
PUSH DS	; 由于在当前数据段中传送数据, 所以使
POP ES	; ES等于DS
MOV SI, 1000H	; 置源串指针初值
MOV DI, 2000H	; 置目的串指针初值
MOV CX, 100	; 重复次数
REP MOVSB	; 每次传送一字节数据

...

从串的低位
开始传送

3.4.4 重复前缀REPZ/REPE

格式: **REPZ** 串操作指令

或 **REPE** 串操作指令

功能: 重复后面的串操作指令,

每重复一次**CX-1**, 重复一

直进行到**CX为0** 或串操作指令

使零标志**ZF为0** 止

- **REPZ/REPE**前缀可以理解为:
- 当数据串没有结束 (**CX≠0**),
- 并且串相等 (**ZF=1**), 则继续比较

例: 检测一段被传送过的数据 (字节串) 是否与源串完全相同, 若不同找到该字节的位置。

```
LEA SI, [1000H]
```

```
LEA DI, [2000H]
```

```
MOV CX, 20H
```

```
CLD
```

```
REPE CMPSB
```

```
JCXZ A11
```

```
DEC SI
```

```
DEC DI
```

```
JMP B11
```

```
A11: MOV SI, 0
```

```
MOV DI, 0
```

```
B11: HLT
```

3.4.4 重复前缀REP**NZ**/REP**NE**

格式: **REP**NZ**** 串操作指令 或 **REP**NE**** 串操作指令

功能: 重复后面的串操作指令, 每重复一次**CX-1**, 重复一直进行到**CX为0** 或串操作指令使零标志**ZF为1** (相等) 止

➤ **REP**NZ**/REP**NE**** 前缀可以理解为: 当数据串没有结束 (**CX≠0**), 并且串不相等 (**ZF=0**), 则继续比较

例:
在某字符串中查找是否存在“\$”字符

```
MOV  DI, 1000H ; 假设目的串的偏移是1000H
MOV  AL, '$'    ; 要找的关键字送AL
REPNE SCASB     ; 扫描查找直到CX=0或 ZF=1
AND  CX, 0FFFFH ; 如果CX=0表示没找到“$”
JZ   没找到
```

没找到: ...

总结：串操作

在程序设计时应掌握以下三个要点：

- (1) 利用方向标志DF设定串操作中地址修改的方向
(用 **STD**, **DF=1**或**CLD**, **DF=0**)。
- (2) 利用**DS:SI**和**ES:DI**设定源串和目标串的首地址。
- (3) 利用**CX**设定被处理数据串的字节个数或字个数。
- (4) 串处理指令每次只处理一个单元的数据(**字节或字**)，为了能执行多达**64KB**的串操作，8086提供了重复操作指令。根据不同的重复条件，可选择下列三种形式之一。

REP ; $(CX)-1=(CX)\neq 0$ ，重复执行，
 $(CX)=0$ ，停止。

REPE/REPZ ; $(CX)-1=(CX)\neq 0$ ，且 $ZF=1$ ，重复行，
 $(CX)=0$ 或 $ZF=0$ 停止。

REPNE/REPNZ ; $(CX)-1=(CX)\neq 0$ ，且 $ZF=0$ ，重复执行
 $(CX)=0$ 或 $ZF=1$ 停止。

表： 串操作指令的重复前缀、操作数和地址指针

指 令	重 复 前 缀	操 作 数	地址指针寄存器
MOVS	REP	目的,←—源	ES:DI, DS:SI
LODS	无	AL/AX←— 源	DS:SI
STOS	REP	目的←—AL/AX	ES:DI
CMPS	REPE/REPNE	源, -目的	DS:SI, ES:DI
SCAS	REPE/REPNE	AL/AX-目的	ES:DI

串操作指令编程步骤：

- SI=源串偏移地址**
- DI=目的串偏移地址**
- CX=串长**
- DF标志置位/清零**
- 执行串操作指令**

【例1】 将首地址为1000:2000H的源字符串传送到以3000:4000H为首地址的内存区，字符串的长度为N个字节。

CLD

MOV AX,1000H

MOV DS,AX

MOV SI,2000H

MOV AX,3000H

MOV ES,AX

MOV DI,4000H

MOV CX,N

REP MOVSB

【例2】 比较两个字符串是否在相同位置有相同的元素，它们的源串和目的串的地址分别为DS段内符号地址SRC和ES段内符号地址DST，字符串的长度为N个字节。

CLD

LEA SI, SRC

LEA DI, DST

MOV CX, N

REPNE CMPSB ; 重复比较字符串

JNE NOT-FOUND ; 无相同的元素，转NOT-FOUND，

DEC SI 否则继续执行下一条指令。

DEC DI

【例3】 在首地址ES:DST的字符串中检查是否有字符 “M”，字符串的长度为N个字节。

CLD ; DF=0, 增量方向

LEA DI , DST

MOV CX , N

MOV AL , ‘M’ ; 搜索字符送AL

REPNE SCASB ; 重复搜索字符串是否有字符 ‘M’

JNE NOT-FOUND

DEC DI

3.5 控制类指令

3.5.1 程序转移的地址

不在同一段了

程序的寻址 { 代码段寄存器 **CS**
程序指针寄存器 **IP** **FAR**

段间转移（段间调用）：改变了段和偏移量（**CS**和 **IP**）

段内转移（段内调用）：只改变偏移量 **IP** **NEAR**

在同一段

直接转移：转移的地址信息直接出现在指令中

间接转移：转移的目标地址信息间接存储于
某寄存器或某一内存单元

相对转移：以当前的**IP**值增加或减少某一个值

绝对转移：以一个新的值完全代替当前的**IP**值

控制转移指令

在8086程序中，指令执行的顺序是由**CS**和**IP**确定的。为了使程序转移到新的地址去执行，可采用改变CS和IP，或者仅改变IP的方法来实现。前者为段间转移(远转移)，属性为**FAR**；后者称为段内转移(近转移)，属性为**NEAR**。

无论是段内转移还是段间转移，都有直接转移和间接转移之分。**直接转移**是转移的目标地址由IP加上指令中的一个8位或16位的位移量；**间接转移**，则是转移目标地址间接存储在某一寄存器或内存单元中。

1. 无条件转移指令

1) 段内直接短转移

格式: **JMP SHORT OPR ;** ($IP \leftarrow OPR$ 的偏移地址)

功能: $IP \leftarrow (IP_{源} + 2) + 8位$ 位移量, 其中8位位移量是CS段内目标地址OPR与JMP指令的下一条指令地址之差, 为一个有符号数, 只允许在-128~+127字节范围内转移, 为相对转移指令。

当向地址增大方向转移时, 位移量为正; 向地址减小方向转移时, 位移量为负。

例如:

...

JMP SHORT HELLO ;

...

...

HELLO: ...

► 在此程序段中, 若JMP指令的当前地址是CS段内0100H ($IP_{源}$), 目标地址是010AH (IP), 因为该指令的机器代码是E908H, 所以

► 8位位移量 = $IP - (IP_{源} + 2)$

► 8位位移量为010AH-0102H=08H

► (其中0102H是JMP指令的下一条指令的地址($IP_{源} + 2$))。

2) 段内直接近转移

格式: **JMP NEAR PTR OPR**

功能: $IP \leftarrow (IP_{\text{源}} + 3) + 16\text{位位移量}$ 。它也是一种相对转移指令。

汇编格式中的OPR可使用符号地址，地址属性NEAR可省略。由于位移量为16位，所以可转移到段内的任何位置。

例如:

...

JMP NEXT ; 可省略NEAR属性说明
($IP \leftarrow \text{NEXT}$ 的偏移地址)

...

NEXT: ...

在此程序段中，若JMP指令的当前地址是CS段内0100H($IP_{\text{源}}$)，目标地址是1000H(IP)，因为该指令的机器代码是EBFD0EH，所以16位位移量 = $IP - (IP_{\text{源}} + 3) = 1000H - 0103H = 0EFDH$

(0103H是JMP指令的下一条指令的地址($IP_{\text{源}} + 3$))。

3) 段内间接转移

格式：JMP REG16/WORD PTR OPR

功能：IP←(EA)，其中EA的值由OPR的寻址方式确定，它可以使用除立即数寻址以外的任何一种寻址方式。如果指定的是16位寄存器，则把寄存器的内容送到IP中；如果指定的是存储器单元地址，则将该地址单元开始的一个字的内容送到IP中去。

例如：在指令中，若(DS)=2000H，(BX)=0100H，
(20125H)=34H，(20126H)=12H，

MOV BX, 0100H

JMP BX ; (IP)=0100H

MOV WORD PTR [BX+25], 1234H

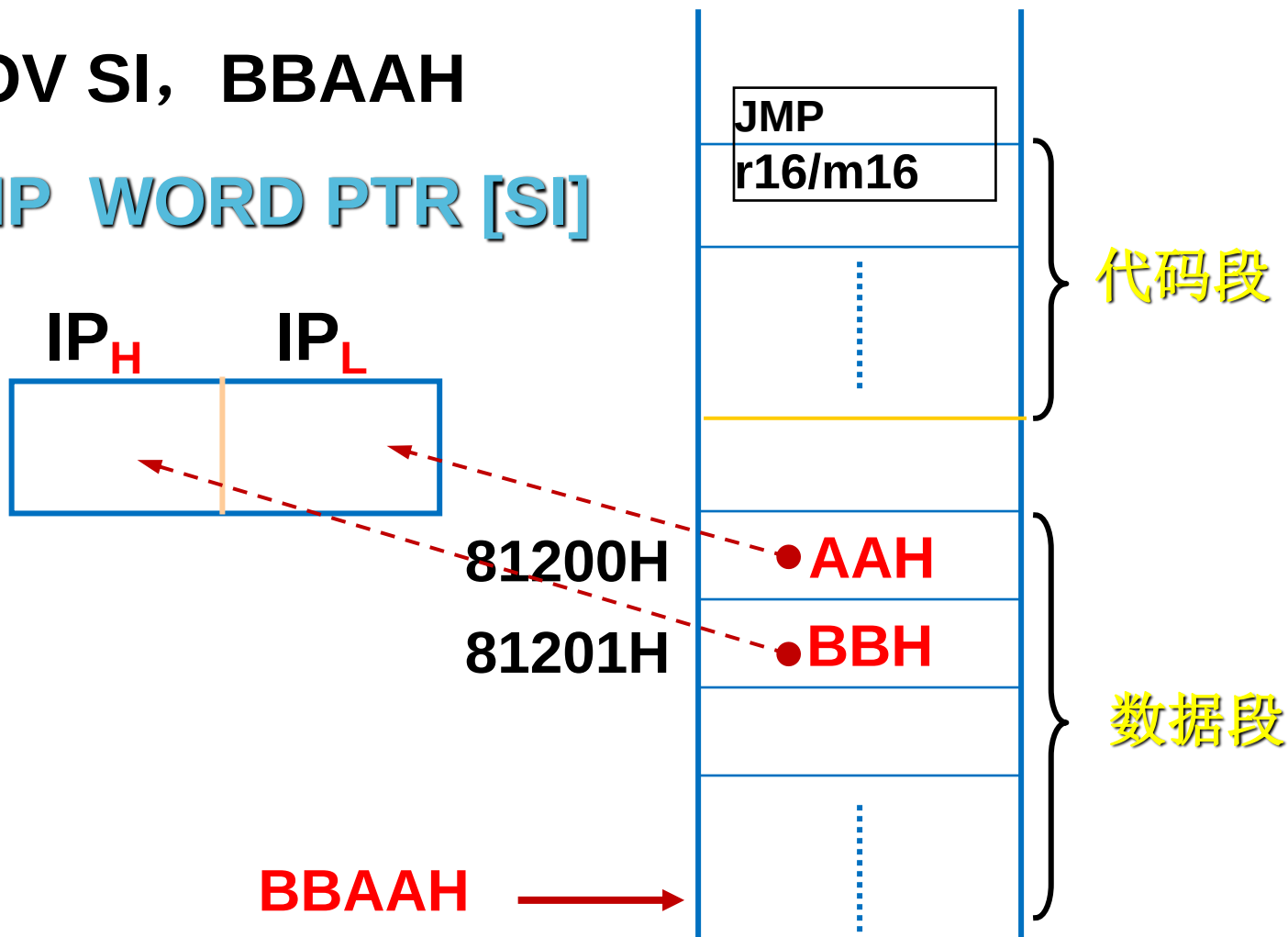
JMP WORD PTR [BX+25H] ; (IP)=1234H

则第一条指令的结果是(IP)=0100H
而第二条指令执行的结果(IP)=1234H。

例: **JMP WORD PTR [SI]** 指令的操作图示:
假定: $(DS) = 8000H$, $(SI) = 1200H$

MOV SI, BBAAH

JMP WORD PTR [SI]



4) 段间直接转移

格式: **JMP FAR PTR OPR**

功能: $IP \leftarrow OPR$ 的段内偏移地址

$CS \leftarrow OPR$ 所在段的段地址

其中**OPR**在汇编格式中可使用**代码段**的符号地址, 如果用数值表达式则要指定转移目标的偏移地址和段地址。这也是一种绝对转移指令。

【例1】OPR用符号地址表示。

CODE1 SEGMENT

...

JMP FAR PTR PROG

...

CODE1 ENDS

CODE2 SEGMENT

...

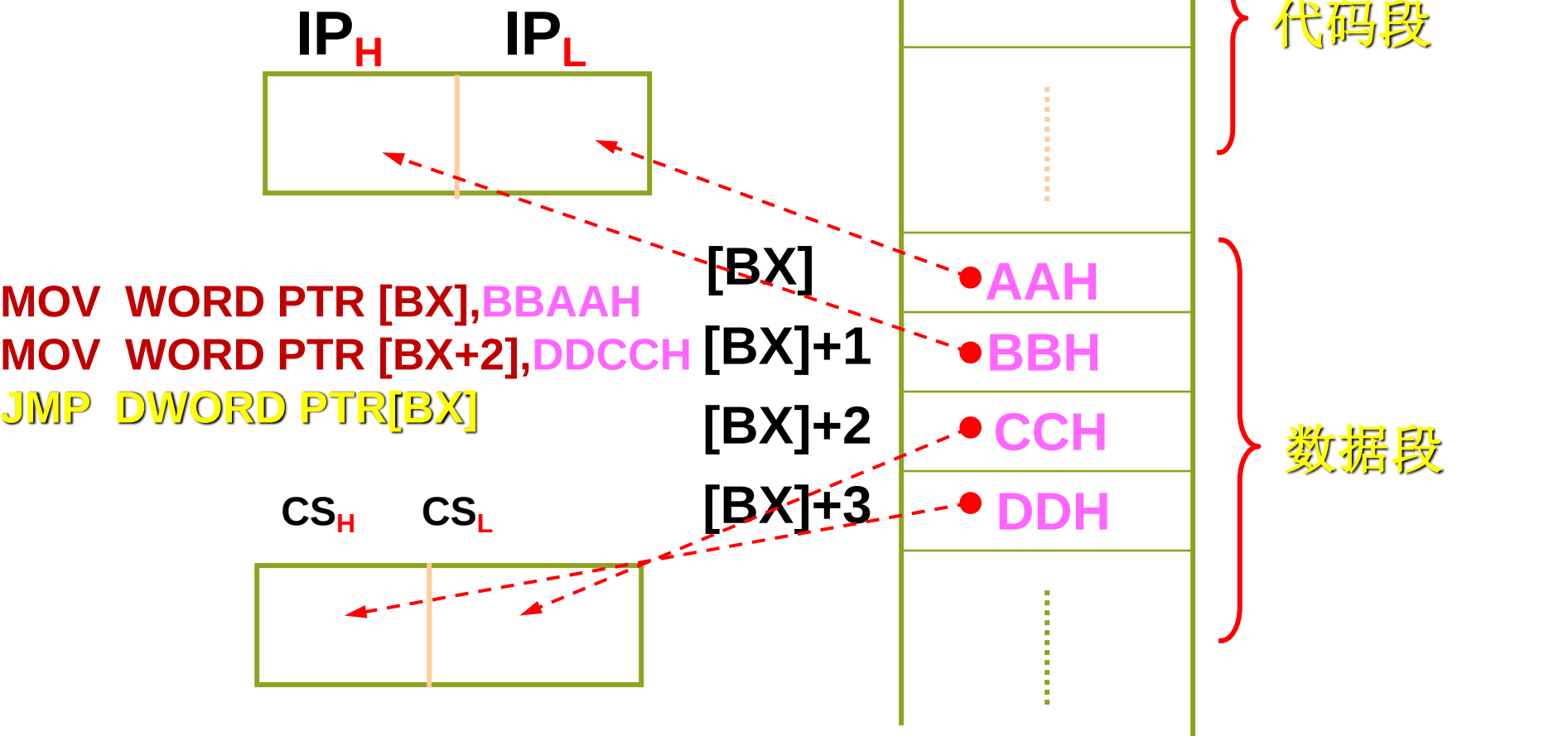
PROG: ... ; $IP \leftarrow \text{PROG}$ 的偏移地址

...

; $CS \leftarrow \text{PROG}$ 的段地址

CODE2 ENDS

例:跳转程序地址在
DDCCH:BBAAH



段间间接调用示意图

小结:

➤ 段内转移——近转移 (near)

- 在当前代码段64KB范围内转移 (±32KB范围)
- 不需要更改CS段地址,
- 只要改变IP偏移地址

➤ 段内转移——短转移 (short)

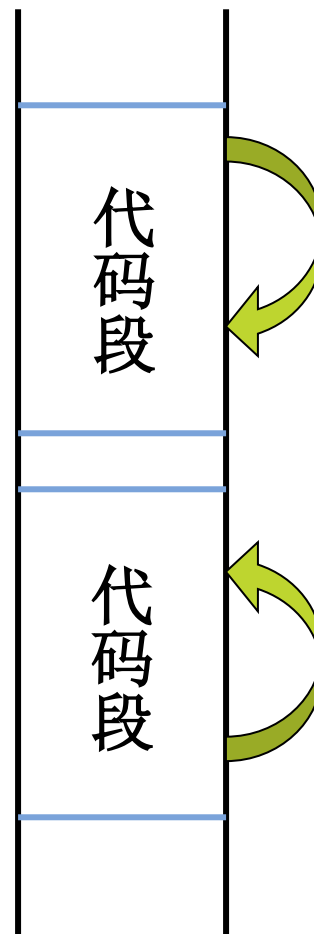
- 转移范围可以用一个字节表达, 在段内-128~+127范围的转移

例: JMP 0120H ; 直接转向0120H

JMP SHORT LP ; 转向LP

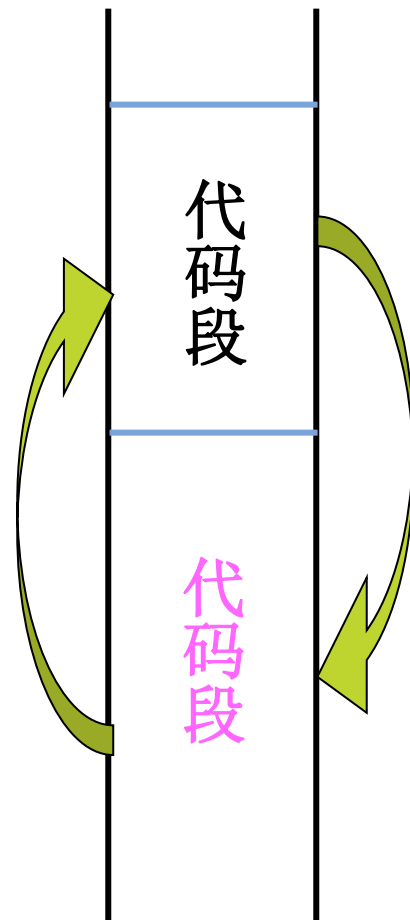
JMP NEAR PTR BBB ; 转向BBB

由于是段内转移, 故转移后CS内容保持不变



➤ 段间转移——远转移（FAE）

- ✓ 从当前代码段跳转到另一个代码段，可以在1MB范围
- ✓ 需要更改CS段地址和IP偏移地址
- ✓ 目标地址必须用一个32位数表达，叫做32位远指针，它就是逻辑地址



- 实际编程时，汇编程序会根据目标地址的距离，自动处理成短转移、近转移或远转移
- 程序员可用操作符**SHORT**、**NEAR PTR** 或**FAR PTR** 强制

JMP SHORT PTR L1

段内直接短转移

JMP NEAR PTR L2

段内直接近转移

JMP BX

JMP WORD PTR [BX]

段内间接近转移

JMP FAR PTR L2

段间直接远转移

JMP DWORD PTR [BX]

段间间接远转移

3.5.2 条件转移指令

条件转移指令以标志位的状态或标志位的逻辑运算结果作为转移依据，若**满足转移条件**，则程序**转移到目标地址**所指示的指令执行；**否则，继续执行下一条指令**。转移地址的偏移量限制在 $-128B \sim +127B$ 范围内，是相对转移。

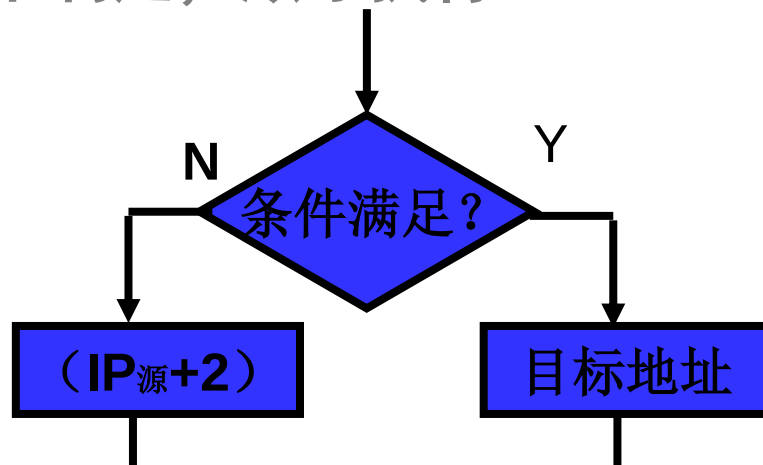
格式：JXX label

功能：XX条件满足，发生转移： **$(IP \leftarrow \text{label的偏移地址})$**

$IP \leftarrow IP_{\text{源}+2} + 8\text{位位移量}$

条件不满足，顺序执行： $IP \leftarrow IP_{\text{源}+2}$

测试无符号
数的高低
Above
Below



测试带符号
数的大小
Greater
Less

3.5.2 条件转移指令

条件转移不影响标志

指令格式	转移条件	转移说明	其他说明
JA/JNBE 标号 JAE/JNB 标号 JB/JNAE 标号 JBE/JNA 标号	CF=0且ZF=0 CF=0或ZF=1 CF=1且ZF=0 CF=1或ZF=1	高于/不低于也不等于 转移 高于或等于/不低于 转移 低于/不高于也不等于 转移 低于或等于/不高于 转移	两个标志 无符号数
JC 标号 JNC 标号 JE/JZ 标号 JNZ/JNE 标号	CF=1 CF=0 ZF=1 ZF=0	进位位为 1 转移 进位位为 0 转移 等于/结果为 0 转移 不等于/结果不为 0 转移	单个标志
JO 标号 JNO 标号 JNP/JPO 标号 JP/JO 标号 JNS 标号 JS 标号	OF=1 OF=0 PF=0 PF=1 SF=0 SF=1	溢出 转移 不溢出 转移 奇偶位为 0 /奇偶位为奇 转移 奇偶位为 1 /奇偶位为偶 转移 符号标志位为 0 转移 符号标志位为 1 转移	位条件转移

1) 根据单个标志位的状态判断转移的指令

例如：

ADD AX,BX

JC TOO_BIG；若加法有进位转至TOO_BIG处理

SUB AL,BL

JZ ZERO ；若减法结果为0，转至ZERO处理

2) 根据两个无符号数的比较结果判断转移的指令

指令(JA, JAE, JB, JBE)，根据两个无符号数相比较所产生的状态标志CF及ZF决定是否转移。

例如：比较无符号数FEH和01H的大小，执行下面的指令后，将转移到ABOVE处继续执行指令。

MOV AL,OFEH

CMP AL,01H

JA ABOVE

3) 根据两个有符号数的比较结果判断转移指令

根据两个有符号数相比较所产生的状态标志SF，OF以及ZF决定是否转移。

例：比较有符号数FEH和01H的大小，执行下面的指令后将不转移到GREATER，而是继续执行JG下面的一条指令，因为有符号数0FEH小于01H。

```
MOV AL,0FEH
```

```
CMP AL,01H
```

```
JG GREATER
```

例：计算 $|X - Y|$ （绝对值）X和Y为存放于X单元和Y单元的16位操作数，结果存入RESULT。

```
MOV AX, X
```

```
SUB AX, Y
```

```
JNS NON
```

```
NEG AX
```

；NEG是求补指令

```
NON: MOV RESULT, AX
```

小结:根据利用的标志位不同,17条指令分成4种情况:

- (1) 判断单个标志位状态;
- (2) 比较无符号数高低;
- (3) 比较有符号数大小;
- (4) 判断计数器CX为0。

例:记录BX中1的个数

XOR AL,AL; AL=0, CF=0

again: CMP BX,0

JE next

SHL BX,1

JNC again

INC AL

JMP again

next: ... ; AL

XOR AL,AL ; AL=0, CF=0

again: CMP BX,0

JZ next

SHL BX,1 ; 也可使用 SHR BX,1

ADC AL,0

JMP again

next: ... ; AL保存1的个数

3.5.3 循环控制指令（1）

1. 计数循环指令LOOP

格式：LOOP 标号

功能：寄存器CX的值减1，如果结果不等于0，则循环至标号，否则顺序执行。

{ DEC CX
JNZ 标号

2. 等于/结果为0循环指令LOOPE/LOOPZ

在该指令前一般要有比较指令CMP

格式：LOOPE 标号 或 LOOPZ 标号

功能：寄存器CX的值减1，且判断只有在CX≠0，而且标志ZF=1的条件下，才循环至标号，否则顺序执行。

3.5.3 循环控制指令（2）

3. 不等于/结果不为0循环指令LOOPNE/LOOPNZ

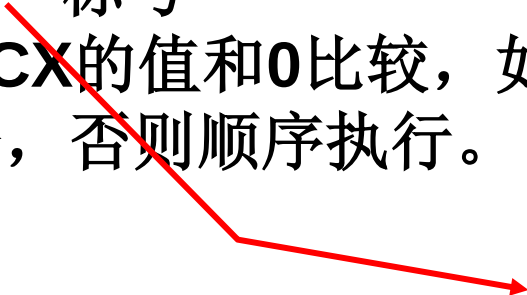
格式: **LOOPNE** 标号 或 **LOOPNZ** 标号

功能: 寄存器**CX**的值减**1**, 且判断只有在**CX≠0**, 而且标志**ZF=0**的条件下, 才循环至标号, 否则顺序执行。

4. CX为0循环指令JCXZ

格式: **JCXZ** 标号

功能: 寄存器**CX**的值和**0**比较, 如果结果等于**0**, 则循环至标号, 否则顺序执行。



```
CMP CX,0  
JZ next
```

总结： 循环控制指令

循环指令用于控制程序的重复执行，它们以CX寄存器为计数器，在其中预置程序的循环次数，并根据对CX内容的测试结果来决定程序是循环至目标地址，还是顺序执行循环控制指令的下一条指令。循环控制指令类似于条件转移指令，也是按给定的条件是否满足来决定程序的执行路径，并且循环控制指令的目标地址必须在其下一条指令第一字节地址的 $-128\text{B} \sim +127\text{B}$ 范围内。因此这也是一种段内直接短转移指令。

按控制循环的方式，循环指令有以下4种格式：

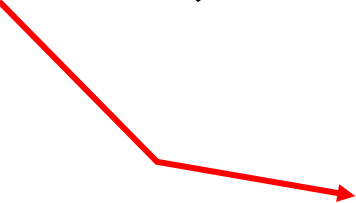
LOOP ; (CX) $\neq$ 0, 则循环
LOOPZ/LOOPE ; ZF=1且(CX) $\neq$ 0, 则循环
LOOPNZ/LOOPNE ; ZF=0且(CX) $\neq$ 0, 则循环
JCXZ ; (CX)=0, 则转移

在这些指令中，除JCXZ指令外，其余都是先使CX寄存器内容减1，然后判断CX的值是否为0。然后测试是否满足测试条件，决定转移与否。

例: **MOV CX,100**
again: **MOVSB** ; 传送一个字节
 DEC CX ; 传送次数减1
 JNZ again ; 判断传送次数CX是否为0
 不为0 (ZF=0), 则转移;
 否则, 结束

 MOV CX,100
again: **JCXZ next** ; 判断传送次数cx是否为0
 MOVSB
 DEC CX
 JMP again

next: ...



```
CMP CX,0  
JZ next
```

例：给1A000H开始的256个内存单元均减去1，若发现某个单元减为0则立即退出循环，其后的单元不再减1。

程序段如下：（逻辑地址为1A00:0H）

```
MOV  AX, 1A00H
MOV  DS, AX          ; 1A00H段
MOV  DI, -1
MOV  CX, 256
GOON: INC  DI
      DEC  BYTE PTR[DI]
      LOOPNZ GOON
      HLT
```

例：在8000H开始的长度为1000字节的字符串中 查找‘S’，若找到，把其偏移地址记录在ADDR中，否则ADDR单元置为0FFFFH。

CLD

MOV DI, 8000H

MOV CX, 1000

MOV AL, 'S'

MOV ADDR, 0FFFFH

GOON: SCASB

LOOPNZ GOON

JNZ DONE

DEC DI

MOV ADDR, DI

DONE: HLT

3.5.4. 调用和返回指令

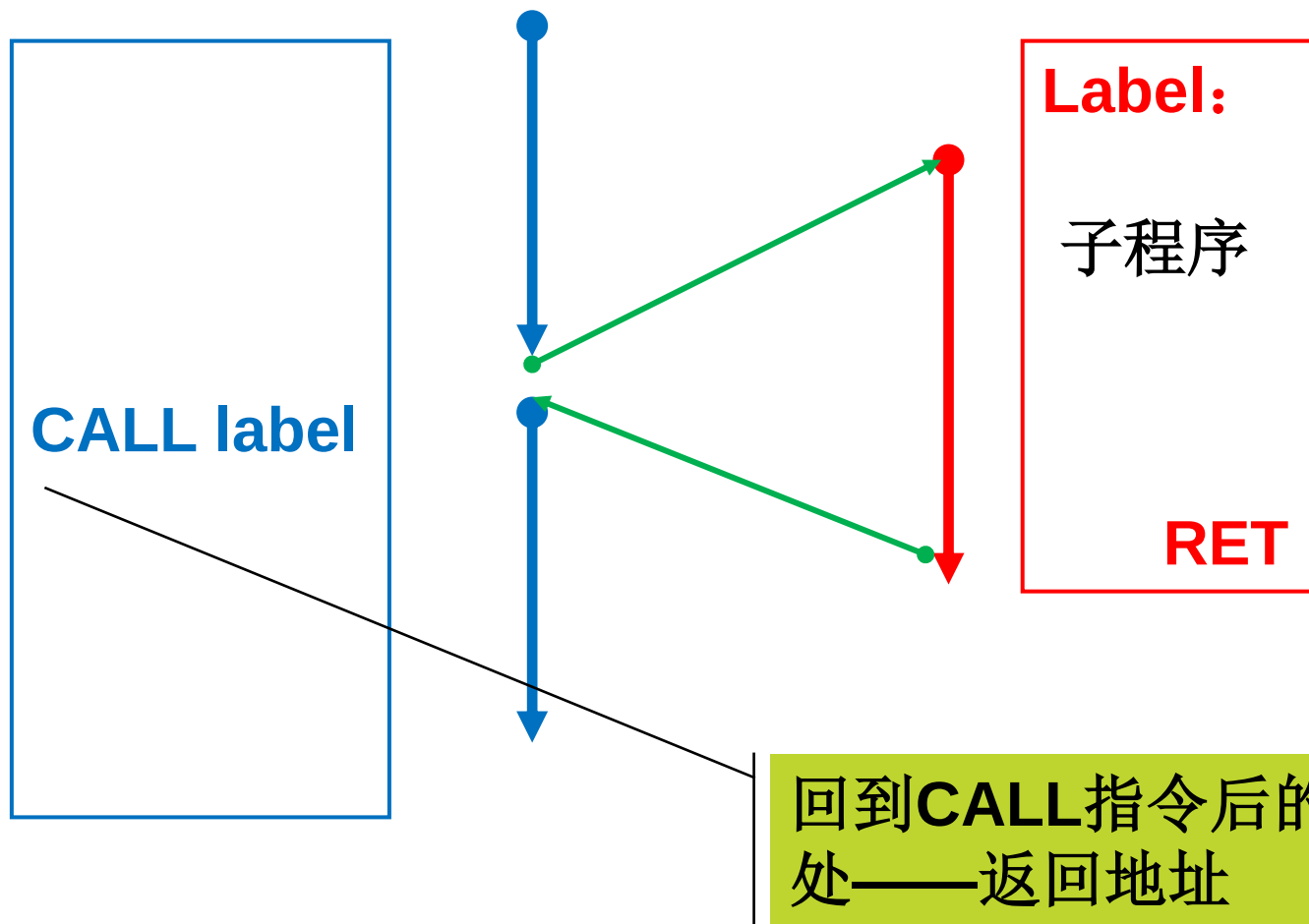
1)调用指令CALL

执行该指令转向目标地址所指示的过程，并且这个过程执行完毕后，仍**返回到CALL指令的下一条指令地址取指令继续执行原来的程序**，因而叫过程调用指令。

当该指令执行时，机器**首先将断点(IP或CS: IP)压入堆栈**，然后以新的目标地址(即过程的首地址)装入IP或CS: IP，于是控制转移到被调用的过程。与**JMP**一样，**CALL**指令也有**段内和段间的直接调用与间接调用**。调用过程也称为调用子程序。

➤ 主程序与子程序

主程序



(1) 段内直接调用。

格式: CALL NEAR PTR **DST**

功能: $(SP) \leftarrow (SP) - 2$; $SP - 2$
 $((SP) + 1, (SP)) \leftarrow IP$; IP入栈
 $IP \leftarrow (IP) + 16\text{位位移量}$; 转向子程序入口

第一步是把子程序返回地址(即CALL指令的下一条指令地址)压入堆栈中, 以便过程返回时使用;

第二步操作 $IP \leftarrow (IP_{\text{源}} + 3) + 16\text{位}$ 则是转移到被调用过程的入口地址去继续执行。指令中的**DST**为过程名, 16位位移量是被调用过程入口地址与CALL指令的下一条指令地址之间的差值。

例: CALL label; $(IP \leftarrow \text{label的偏移地址})$ $IP \leftarrow (IP_{\text{源}} + 3) + 16\text{位}$

段内返回指令RET

格式: RET

功能: $IP \leftarrow ((SP) + 1, (SP))$
 $SP \leftarrow (SP) + 2$

例:位移量由汇编程序在汇编时进行计算,如下:

CS:0102 CALL 0120H ;3字节

CS:0105

则位移量为: 0120-0105H=001BH

于是CALL 0120H的机器码为E8 1B 00

CS:0102	E8	}	CALL 0120H
CS:0103	1B		
CS:0104	00		
CS:0105			

(2) 段内间接调用。

格式：CALL NEAR PTR **DST; (r16/m16)**

功能：SP←(SP)−2 ; SP−2

((SP)+1, (SP))←IP ; IP入栈

IP←(EA) ; 转向子程序入口

该指令与段内直接调用类似，区别在于IP的值不是CALL指令的下一条指令地址加上16位位移量，如果**DST**是**16位通用寄存器操作数**，则把寄存器内容**送IP**；如果**DST**是**字存储器操作数**，则把字存储单元之内容**送IP**。

例如：MOV BX, 1234H

CALL BX ; BX内容送IP=1234H

MOV WORD PTR[BX], 1234H

CALL WORD PTR[BX] ; (DS:BX)中读出一个字数据送
IP=1234H

段内返回指令RET

格式：RET

功能：IP←((SP)+1, (SP))

SP←(SP)+2

CALL指令的操作例：

例：子程序地址在CS:BBAAH

CALL WORD PTR[SI]

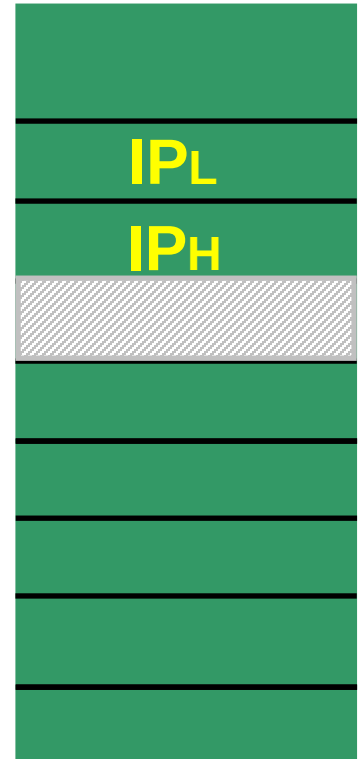
断点：CS:IP

1. 保护断点

执行CALL
指令后

SP=11FE

堆栈



CALL WORD PTR [SI]指令的操作图示:

假定: (DS) = 8000H, (SI) = 1200H

MOV SI,1200H

CS:IP

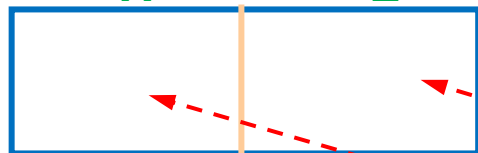
MOV WORD PTR [SI],0BBAAH

CALL WORD PTR [SI]

IP_H

IP_L

BBAAH



81200H

81201H

CALL

NEAR PTR
DST

子程序

RET

• AAH

• BBH

2.得到子程序地址

代码段

数据段

CALL指令的操作例:

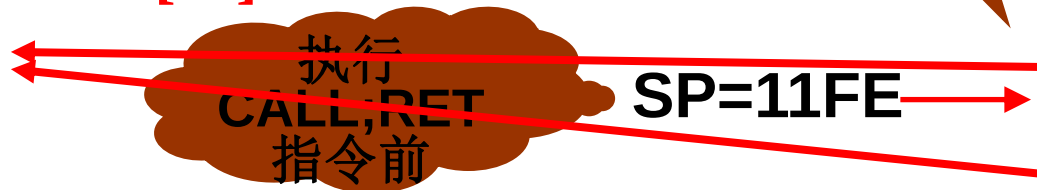
例:子程序地址在CS:BBAAH

CALL WORD PTR[SI]

断点:CS:IP

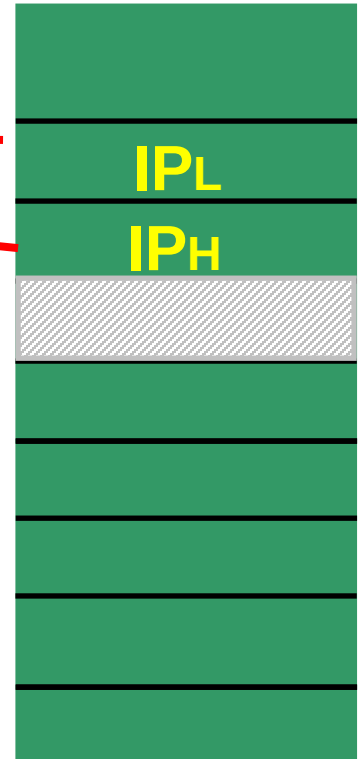
3.返回断点

堆栈



SP=1200

执行CALL;RET指令后



(3) 段间直接调用。

格式：CALL FAR PTR DST

功能：SP←(SP)−2 ； SP−2

((SP)+1, (SP))←(CS) ； CS入栈

SP←(SP)−2 ； SP−2

((SP)+1, (SP))←(IP) ； IP入栈

IP←偏移地址 ； IP由指令中偏移地址取代

CS←段基址 ； CS由指令中的段基址取代

该指令的操作与段内直接调用类似，不同的是：其一，调用时不仅要保护偏移地址，还要保护段地址，顺序是先**CS**入栈，再把**IP**压栈；其二，**CS**和**IP**的值不是用**CALL**指令的下一条指令地址加上位移量取代，而是直接将**DST**的段地址和偏移地址送入**CS: IP**。例如：

CALL 1000H:0100H ； 1000H送CS， 0100H送IP

CALL FAR PTR SUB-PROC

； 过程SUB-PROC的段地址送CS， 偏址送IP

(4) 段间间接调用。

格式: CALL **FAR PTR DST**

功能: $SP \leftarrow (SP) - 2$
 $((SP) + 1, (SP)) \leftarrow (CS)$
 $SP \leftarrow (SP) - 2$
 $((SP) + 1, (SP)) \leftarrow (IP)$
 $IP \leftarrow (EA)$
 $CS \leftarrow (EA + 2)$

mem32

; $SP - 2$
; CS入栈
; $SP - 2$
; IP入栈
; EA取代IP中原地址
; $EA + 2$ 取代CS中原段基址

段间返回返回指令RET:

格式: RET

$IP \leftarrow ((SP) + 1, (SP))$
 $SP \leftarrow (SP) + 2$
 $CS \leftarrow ((SP) + 1, (SP))$
 $SP \leftarrow (SP) + 2$

RET i16 ; 有参数返回RET
指令可以带有一个立即数i16,
则堆栈指针SP将增加, 即

$SP \leftarrow SP + i16$

这个特点使得程序可以方便地
废除若干执行CALL指令以前
入栈的参数

CALL指令的操作例:

例:子程序地址在DDCCH:BBAAH

1.保护断点

CALL DWORD PTR[DI]

断点:CS:IP

堆栈

SP=11FC

执行CALL
指令后

IPL

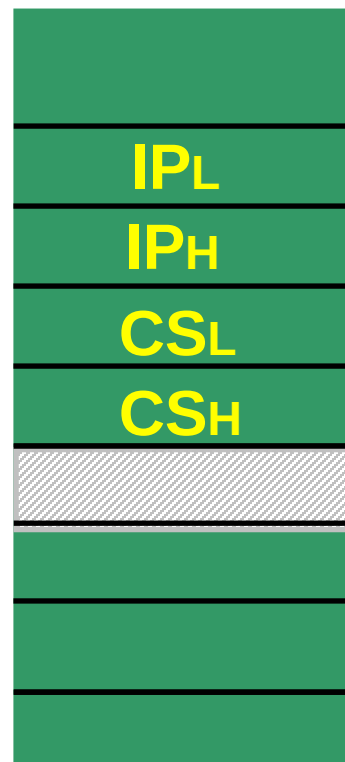
IPH

CSL

CSH

SP=1200

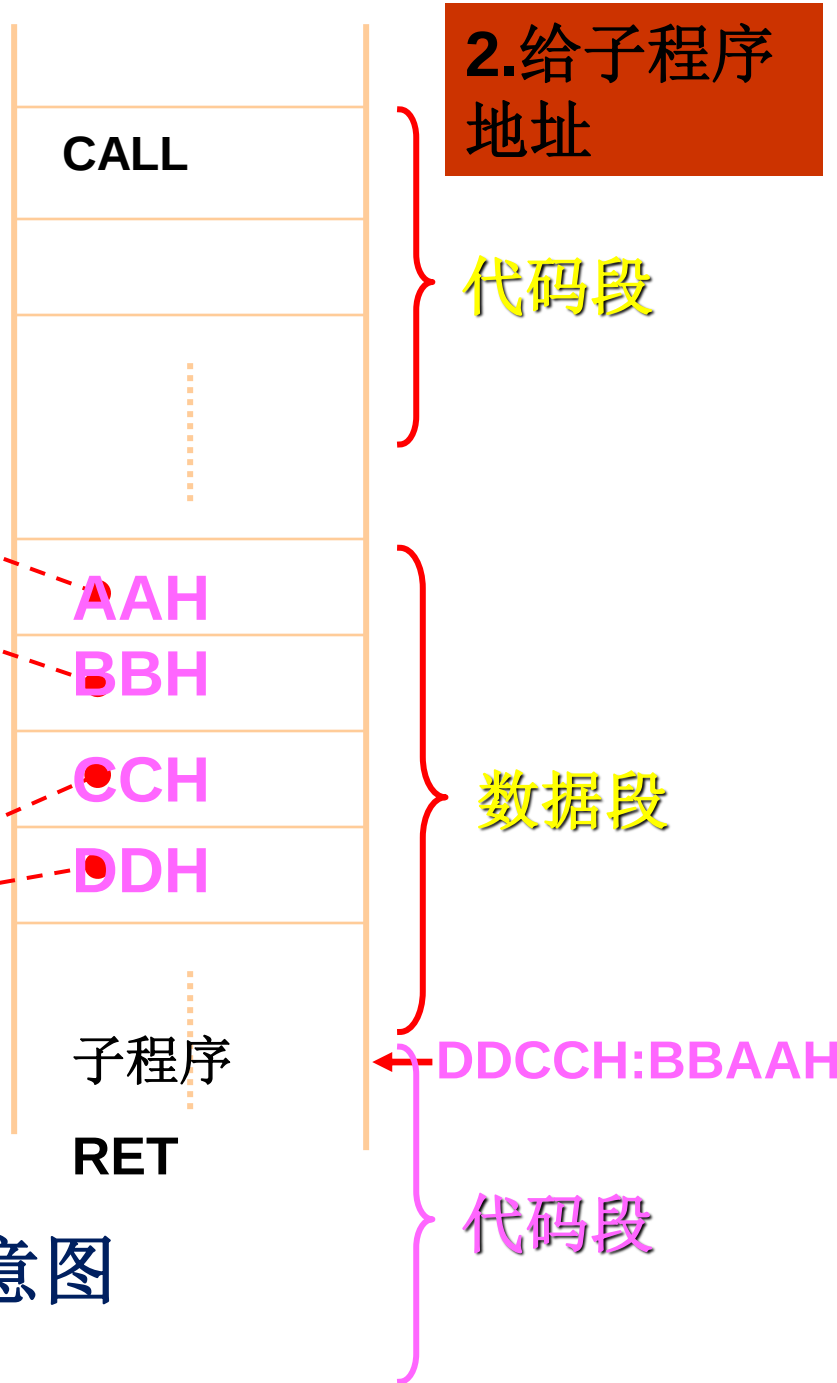
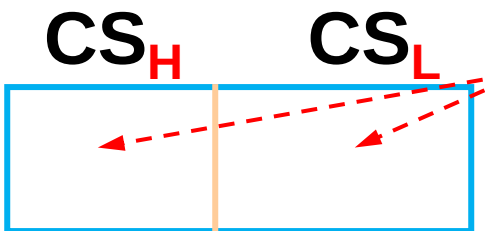
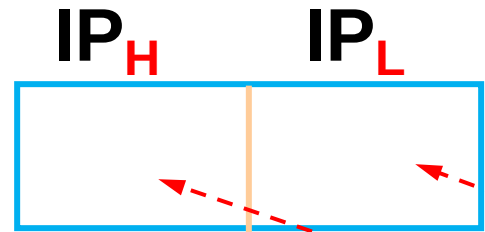
执行CALL
指令前



例:子程序地址在DDCCH:BBAAH

```
CALL DWORD PTR[DI]

MOV WORD PTR [DI],BBAAH
MOV WORD PTR [DI+2],DDCCH
CALL DWORD PTR[DI]
```



段间间接调用示意图

CALL指令的操作例：

例：子程序地址在
DDCCH:BBAAH

CALL DWORD PTR[DI]

断点：CS:IP

3. 返回断点

堆栈

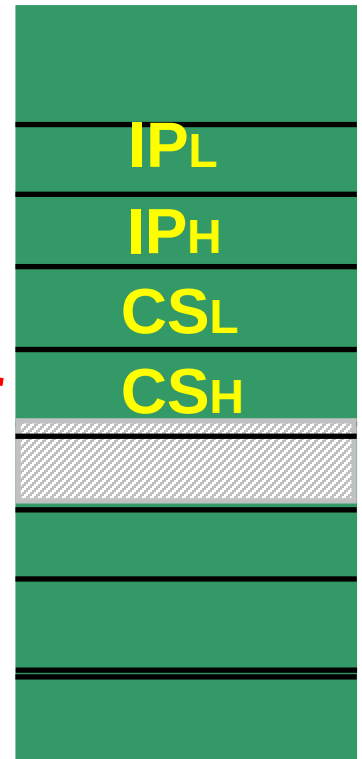
SP=11FC

SP=11FEH

SP=1200

执行CALL
;RET指令
前

执行CALL
;RET指令
后



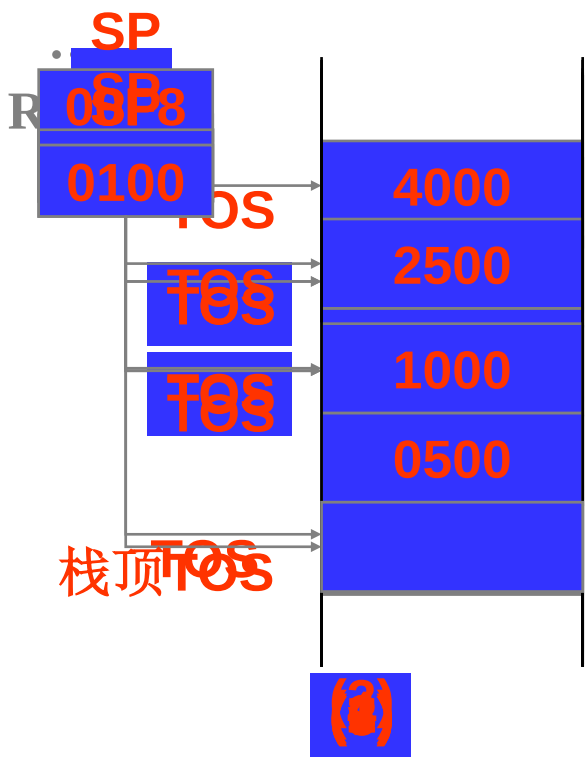
• 例: MAIN

...

...

CALL FAR PTR PRO_A

(CS)=0500H:(IP)=1000H ...



PRO_A: ...

...

...

CALL NEAR PTR PRO_B

(IP)=2500 ...

...

CALL NEAR PTR PRO_C

(IP)=3700 ...

RET

PRO_B: ...

...

CALL NEAR PTR PRO_C

(IP)=4000 ...

...

RET

PRO_C: ...

...

...

...

RET

； 主程序

MOV AL,0CH ； 提供参数AL

CALL htoasc ； 调用子程序

...

； 子程序：将AL低4位的一位16进制数转换成ASCII码

htoasc: AND AL,0FH ； 只取al的低4位

OR AL,30H ； al高4位变成3

CMP AL,39H ； 是0~9，还是0Ah~0Fh

JBE htoend

ADD AL,7 ； 是0Ah~0Fh，加上7

htoend: RET ； 子程序返回

401CH

200AH

例：下面的程序执行后，(AX)=? (DX)=?

CS:2000H MOV AX, 2012H

2003H MOV CX, 200CH

2006H PUSH CX

2007H CALL 4000H

200AH ADD AX, BX

200CH ADD AX, DX ;2012H+200AH=401CH

200EH HLT

... ..

... ..

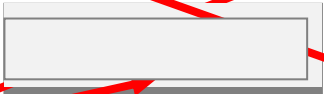
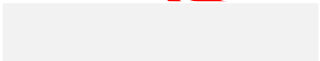
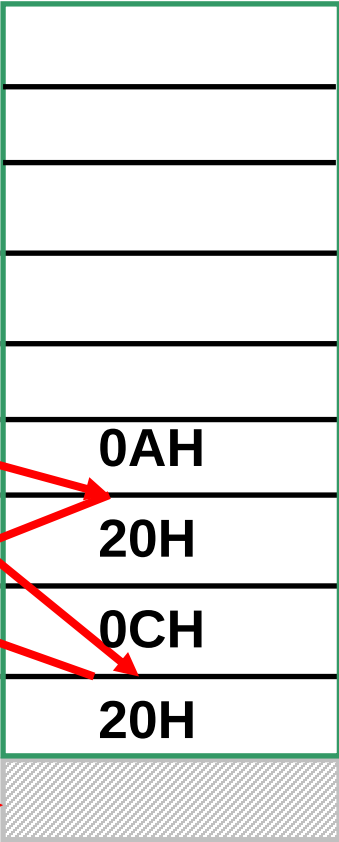
CS:4000H MOV BX, 200AH

POP DX

RET

;DX=200AH

堆栈

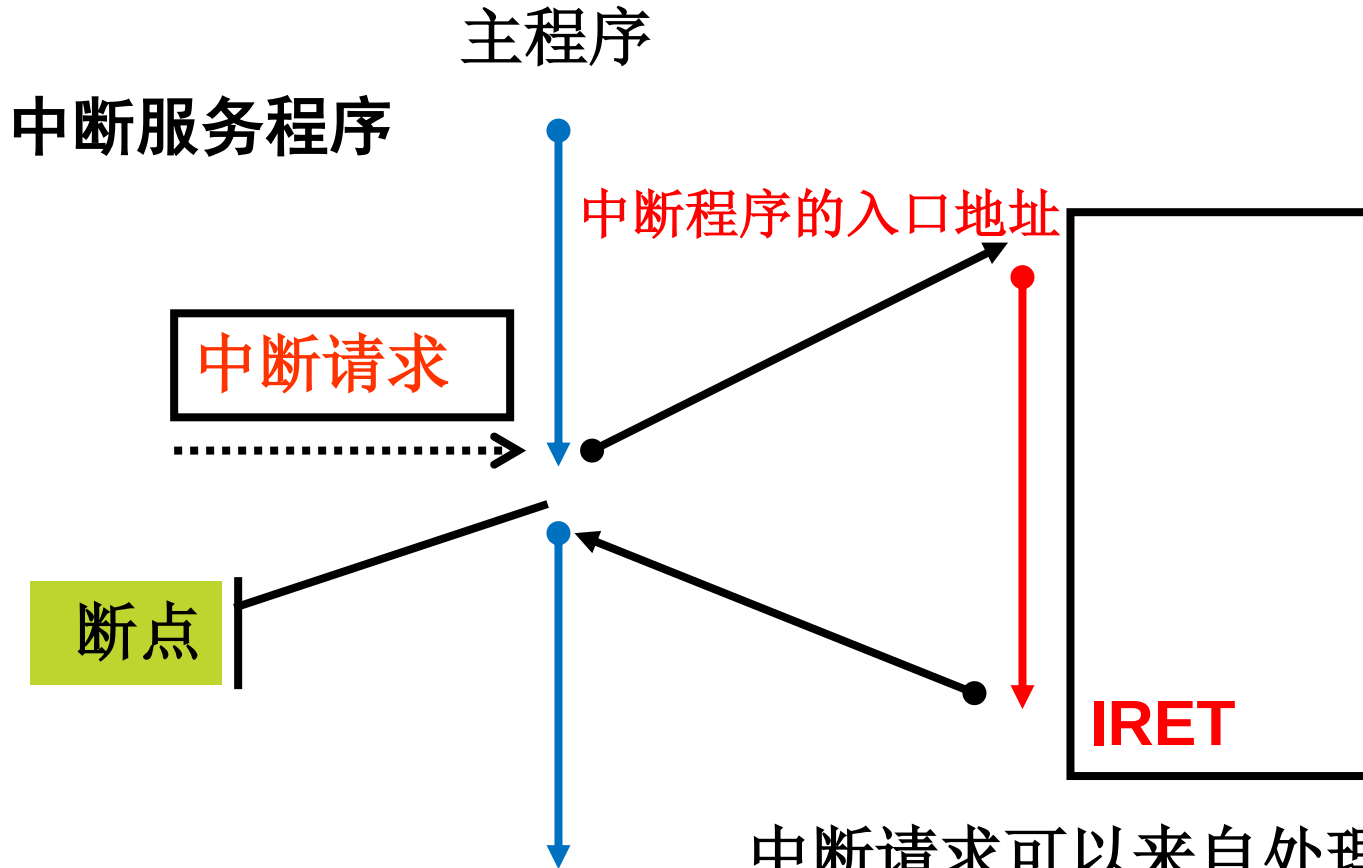


5.5.5. 中断指令

中断和过程调用有些类似，两者都是将返回地址先压栈，然后转到某个程序去执行。它们的区别是：

- (1) **过程调用**转向称为过程的**子程序**，**中断指令**是使控制转向**中断服务子程序**。
- (2) **过程调用**可以是**NEAR**或**FAR**类型，能**直接**或**间接**调用，**中断调用**是段间间接转移到**中断服务程序**。
- (3) **过程调用**只**保护返回地址**，**中断指令**不但**保护返回地址**还要使**标志寄存器进栈**。

中断的过程



中断请求可以来自处理器外部的中断源，
也可以由处理器执行指令引起：
例如执行**INT i8**指令。

中断向量

中断程序的入口地址

➤ 共256种类型,每种占有4个字节单元

00000H	类型0中断处理 程序入口地址	} 类型0的(IP) 和 (CS)
00004H	类型1中断处理 程序入口地址	
00008H	类型2中断处理 程序入口地址	
0000CH		
003FCH	类型0FF中断处理 程序入口地址	} 类型0FF的(IP) 和 (CS)
003FFH		

图 中断向量表

(1) INT n

- 格式:

INT n

中断类型码
 $n = 0 \sim 255$

00000H

内存

0000 : $n \times 4$

中断向量

中断服务程序入口的偏移地址(IP)

中断服务程序入口的段地址(CS)

XXH

XXH

YYH

YYH

- 说明:

$n \times 4 =$ 向量地址。该向量地址中的内容即为中断服务程序入口地址(段:偏移), 入口地址也称为“中断向量”。

8086的外部中断

8086可以管理256个中断

- 各种中断用一个向量编号来区别
- 主要分成外部中断和内部中断
- 外部中断——来自CPU之外的原因引起的中断，又可以分成
 - ✓可屏蔽中断：可由CPU的中断允许标志IF控制
 - ✓非屏蔽中断：不受CPU的中断允许标志IF控制

8086的内部中断(P₁₀₀)

内部中断——CPU内部执行程序引起的中断，又可以分成：

除法错中断：执行除法指令，结果溢出产生的 **0** 号中断
溢出中断指令INTO

格式：INTO

功能：当OF=1，进入溢出中断；当
OF=0，顺序执行下一条指令。

单步中断：TF=1在每条指令执行后产生的 **1** 号中断

非屏蔽中断：用于**非屏蔽中断**（**INT 2**）的 **2** 号中断,由NMI

断点中断：用于断点调试（**INT 3**）的 **3** 号中断,或 **INT**

溢出中断：执行溢出中断指令，OF=1产生的 **4** 号中断

指令中断：执行中断调用指令**INT n=i8**产生的 **n=i8** 号中断

1) 软中断指令INT

格式：INT n

其中n为中断类型号，取值范围0~255，因而可实现256种不同的中断。

功能：SP←(SP)-2 ; SP-2

((SP)+1, (SP))←(FR) ; 标志寄存器进栈

IF←0, TF←0 ; 清中断允许和单步标志

SP←(SP)-2 ; SP-2

((SP)+1, (SP))←(CS) ; CS入栈

SP←(SP)-2 ; SP-2

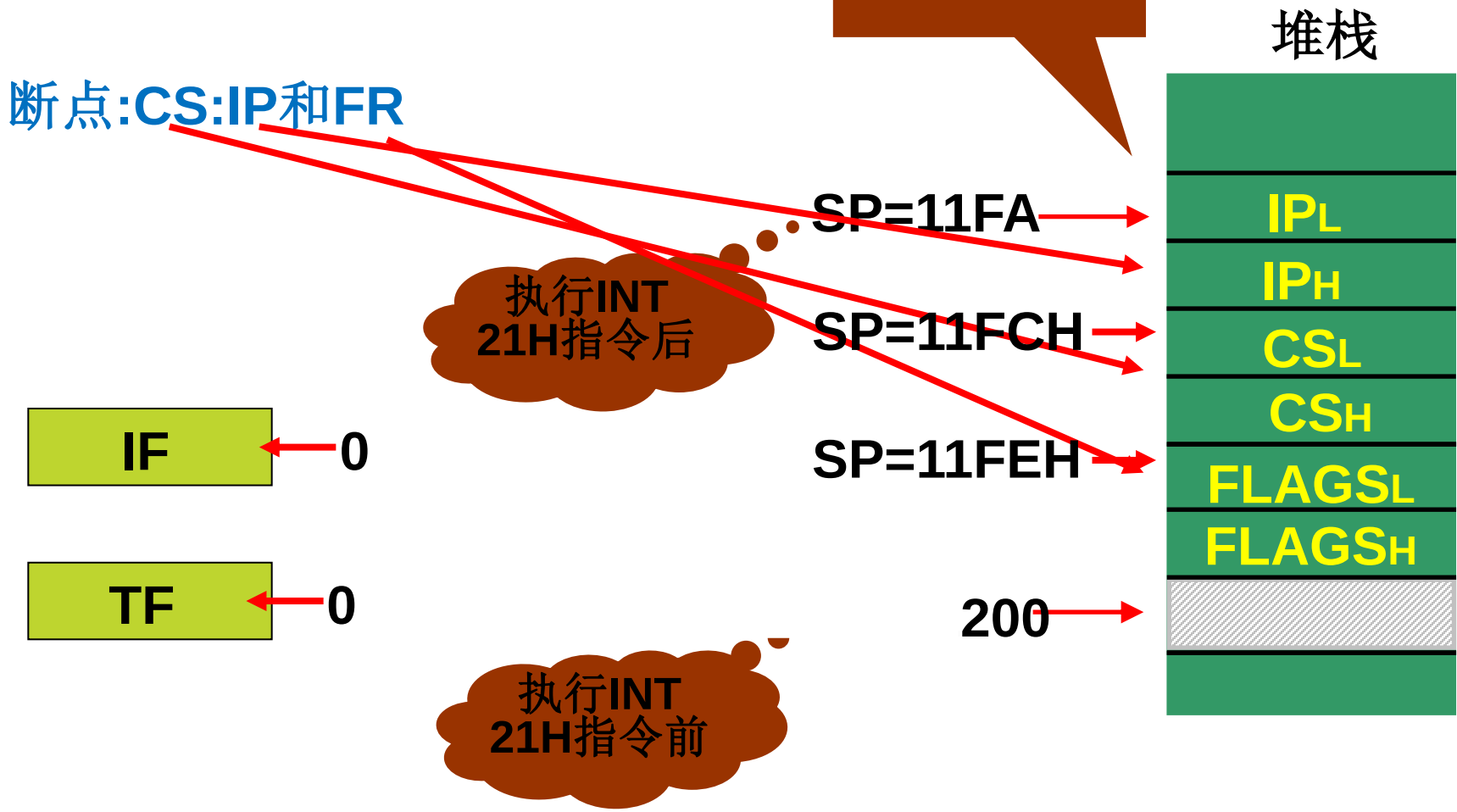
((SP)+1, (SP))←(IP) ; IP入栈

IP←(n*4) ; 取中断服务程序偏移地址，n
为中断类型码

CS←(n*4+2) ; 取中断服务程序的段地址

INT指令的操作例：

➤INT 21H



INT指令的操作例（续）：

- 执行INT 21H指令后， CS=? IP=?

因为 $n=21H$ ，所以 $n \times 4=84H$ 。

下图中， $(0:0084H)=2000H:1123H$

MOV DI,0084H

MOV WORD PTR[DI],1123H

MOV WORD PTR[DI+2],2000H

INT 21H

0000 : $21H \times 4$

0000:0084H

IP

CS

23H

11H

00H

20H

2.给中断服务程序地址

所以：21H中断程序地址为 CS=2000H IP=1123H

2) 中断返回指令 IRET

格式：IRET

功能：与RET指令类似，都是使控制返回到主程序。但是，该指令除了出栈偏移地址和段地址外，还恢复标志寄存器的内容。

$(IP) \leftarrow ((SP)+1, (SP))$; IP断点出栈

$SP \leftarrow (SP)+2$; SP+2

$(CS) \leftarrow ((SP)+1, (SP))$; CS断点出栈

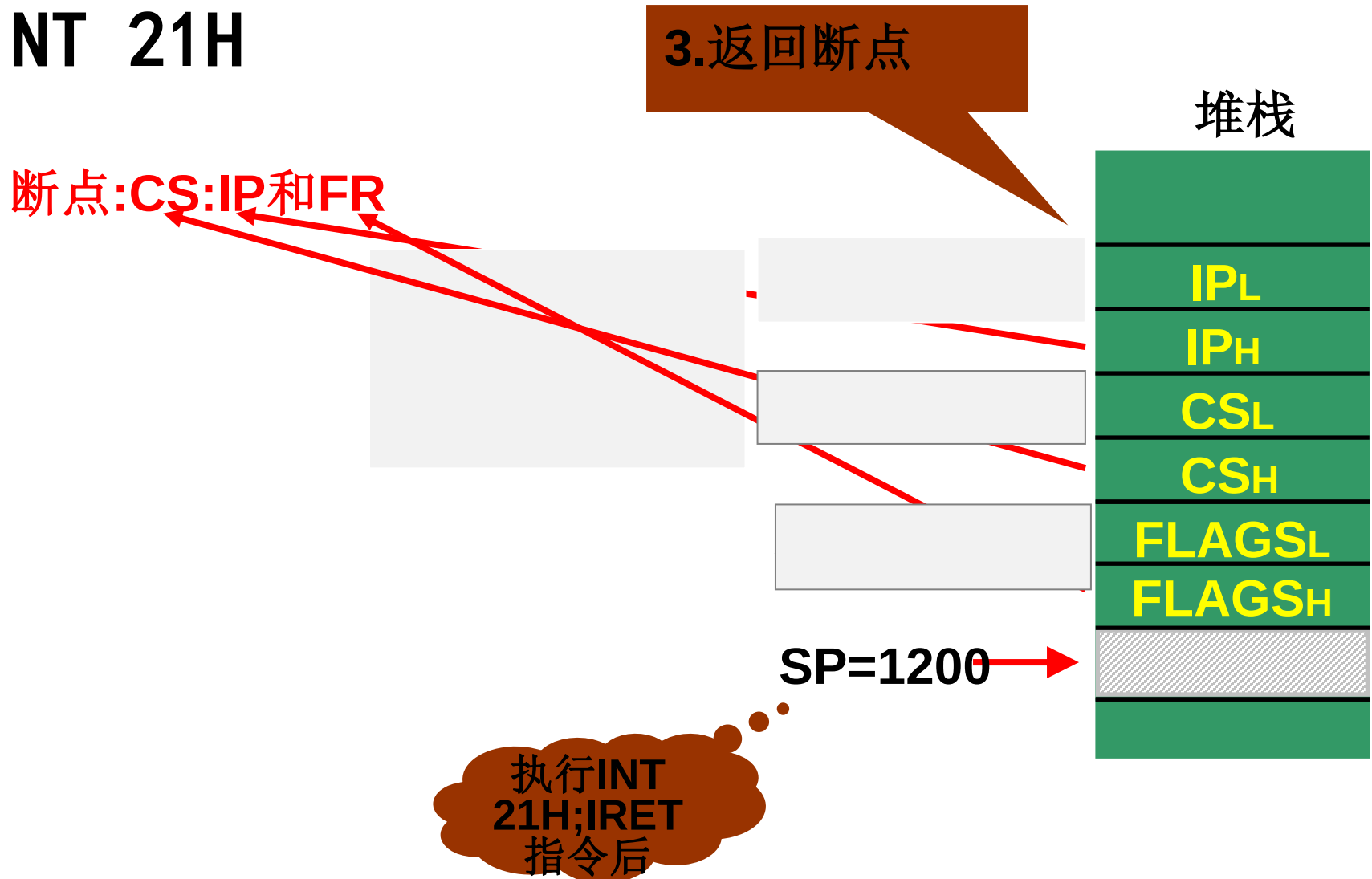
$SP \leftarrow (SP)+2$; SP+2

$(FR) \leftarrow ((SP)+1, (SP))$; 标志寄存器出栈

$SP \leftarrow (SP)+2$; SP+2

INT指令的操作例:

• INT 21H



3.5.6 处理机控制指令（1）

1. 对标志位操作指令

指令格式	功能	说明
CLC	清除进位标志	CF=0
STC	置进位标志	CF=1
CMC	取反进位标志	CF=CF
CLD	清除方向标志	DF=0
STD	置方向标志	DF=1
CLI	清除中断标志	IF=0
STI	置中断标志	IF=1

3.5.6 处理机控制指令（2）

2. 外部同步命令

指令格式	功能
WAIT	使CPU进入等待状态，每隔5个时钟周期，测试一次芯片上的 $\overline{\text{TEST}}$ 状态，直到 $\overline{\text{TEST}}$ 线上的信号变为有效为止，等待状态不做任何操作
ESC	交权指令，把控制权交给协处理器，所以，ESC后面是协处理器指令
LOCK	是一个指令前缀，使得在执行下一条指令期间发出总线封锁LOCK信号，指令执行过程中禁止其它协处理器使用总线

3.5.6 处理机控制指令（3）

3. 其它控制指令

(1) 处理器暂停指令HLT

HLT ； **CPU**进入暂停状态（在暂停状态不进行任何操作）

脱离暂停的条件



- 1.在RESET线上有复位信号
- 2.在NMI线上有请求
- 3.在中断允许情况下，在INTR线上有请求

所以，**HLT**指令常在程序中为了等待中断而使用。

(2) 空操作指令NOP



The end, thank you!